ThreadForge 项目说明

一份按「文件 → 函数 → 变量」逐层展开的代码说明。

目标：任何人（包括三个月后的你自己）读完之后，能在不看源码的情况下说清楚

每个文件为什么存在、每个函数在做什么、每个关键变量由谁写、被谁读、要不要加锁。

C++17 线程池内核（4 484 行） · Dear ImGui 仪表盘 · 无界面 CLI · 22 个单元测试

生成日期：2026-11-06 | 共 15 节 + 6 个附录 | 含 2 张图

目　录

0. 这份文档怎么用 3

1. 项目定位 4

2. 逻辑实现图与运行流程图 5

2.1 逻辑实现图（代码分层与依赖方向） 5

2.2 运行流程图（三个线程角色 + 一次任务的一生） 6

3. 构建、运行与测试 7

3.1 构建 7

3.2 三个可执行文件 7

3.3 界面怎么用 8

4. 目录结构与文件清单 9

5. 逐文件说明 11

5.1 `task.hpp`（238 行）— 任务的数据模型 11

5.2 `config.hpp`（47 行）+ `config.cpp`（77 行）— 配置与校验 14

5.3 `priority_queue.hpp`（79 行）+ `priority_queue.cpp`（158 行）— 优先级队列 16

5.4 `event_log.hpp`（71 行）+ `event_log.cpp`（93 行）— 事件环形缓冲 18

5.5 `worker.hpp`（64 行）+ `worker.cpp`（59 行）— 工作线程的"躯壳" 20

5.6 `metrics.hpp`（70 行）— 体检报告 21

5.7 `simulator.hpp`（45 行）+ `simulator.cpp`（84 行）— 假负载 21

5.8 `pool.hpp`（170 行）— 线程池的对外契约 22

5.9 `src/pool.cpp`（738 行）— 线程池本体 26

5.10 `app/dashboard.hpp`（89 行）+ `app/dashboard.cpp`（673 行）— 仪表盘 32

5.11 `app/theme.hpp`（53 行）+ `app/theme.cpp`（187 行）— 配色与中文字体 36

5.12 `app/main.cpp`（287 行）— 平台层：窗口 + D3D11 + 主循环 37

5.13 `apps/cli_main.cpp`（285 行）— 无界面压测程序 40

5.14 `tests/` — 手写测试框架 + 22 个用例 43

5.15 `CMakeLists.txt`（118 行）— 构建脚本 47

6. 并发约束与踩坑清单 49

6.1 十条硬约束 49

6.2 五个真实踩过的坑 49

6.3 写新代码时的自检清单 50

附录 A：配置项速查（想调什么改哪里） 51

附录 B：实测数据 52

附录 C：一次任务的一生 53

结语 54

0. 这份文档怎么用

你想做的事	直接看
先建立整体印象	第 1 节 项目定位 + 第 2 节 两张图
把项目跑起来	第 3 节 构建、运行与测试
查某个文件干什么	第 5 节 逐文件说明
改代码前先看规矩	第 6 节 并发约束与踩坑清单
调参数	附录 A 配置项速查
看性能数字	附录 B 实测数据
给别人讲这个项目	第 2 节 两张图 + 附录 C 一次任务的一生

约定：

- 行号形如 `src/pool.cpp:391`，指的是**本文档写作时的版本**；代码改动后行号会漂移，函数名不会。
- 「🔒」表示"必须在持有 `mutex_` 时调用"；「🔓」表示"必须不持锁调用"。
- 「易错点」是这个项目真实踩过的坑，不是假想。

1. 项目定位

ThreadForge 是一个**教学用的进阶线程池**：内核 1000 行左右，配套一个实时可视化仪表盘和一个无界面压测 CLI。

它要回答的不是"线程池怎么写"，而是下面这几个问题：

1. 线程数**什么时候变**、变多少、凭什么判断？（动态扩缩容 + 冷却 + 滞回）
2. 高优先级任务会不会把低优先级**饿死**？（优先级老化）
3. 队列满了怎么办？下游一直提交，系统怎么**优雅地拒绝**？（背压）
4. 任务**取消**到底能怎么做？（两段式：排队直接摘、运行中只能协作）
5. 一个任务抛异常，线程池会不会**少一个线程**？（异常隔离）
6. 怎么在不拖慢线程池的前提下，**每帧**把内部状态画到界面上？（值快照）

维度	取值
语言 / 标准	C++17（`std::thread`、`std::mutex`、`condition_variable`、`atomic`）
内核依赖	只有标准库 + `Threads::Threads`，**没有任何平台 API**
界面	Dear ImGui 1.91.9b + Win32 + D3D11（仅 `app/` 需要）
构建	CMake ≥ 3.20 + Ninja；实测工具链 MinGW-w64 g++ 8.1.0
测试	手写框架（无第三方依赖），CTest 驱动，22 个用例
规模	自有代码 4484 行（不含 `third_party/`）

三个可执行文件，同一个内核：

```
threadforge_app.exe   图形仪表盘（Win32 + D3D11 + ImGui）
threadforge_cli.exe   无界面压测：提交 → 观察扩缩容 → 打报告
tp_tests.exe          22 个单元测试（队列 6 + 线程池 16）
```

内核 `tp_core` 不知道"界面"这两个字怎么写：它只提供 `submit / cancel / snapshot / events`。所以换 UI 框架、搬到 Linux、拿去做服务端，内核一行都不用改。

2. 逻辑实现图与运行流程图

2.1 逻辑实现图（代码分层与依赖方向）

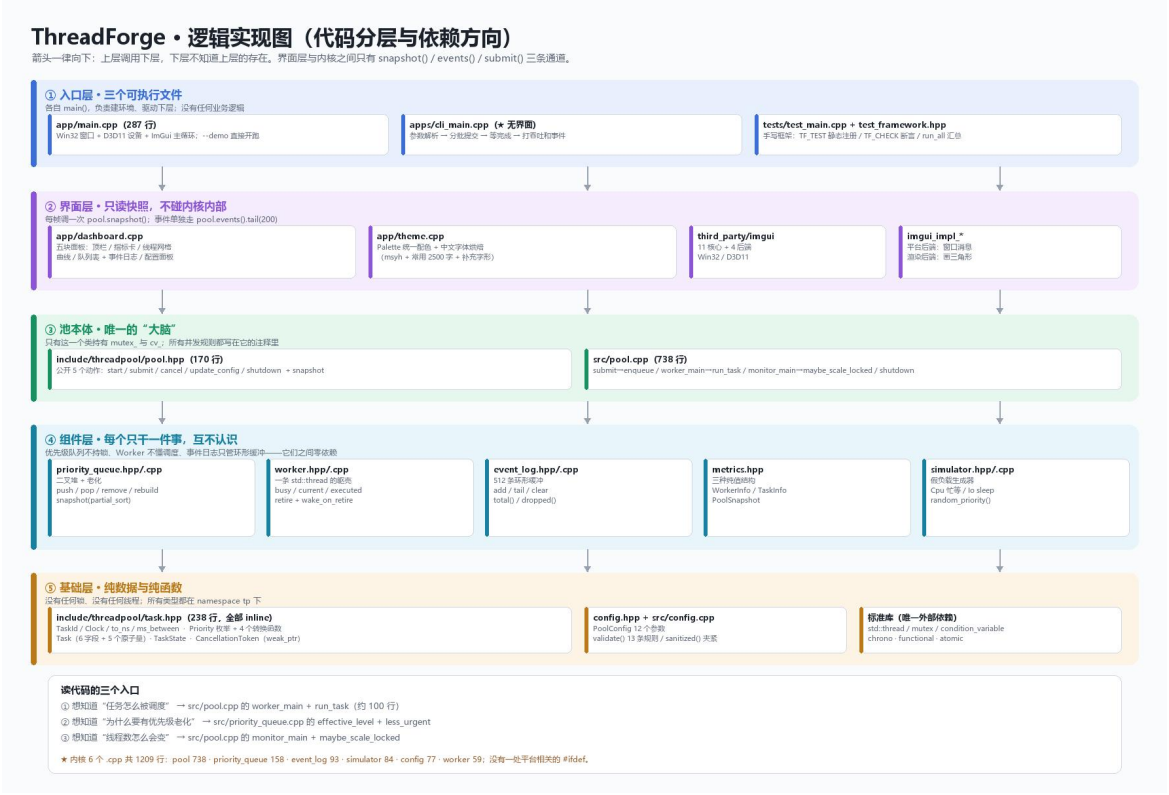


图 1　逻辑实现图

读这张图只要记住三件事：

1. **箭头只往下。**`app/main.cpp` 可以 `#include "threadpool/pool.hpp"`，反过来绝对不行——`tp_core` 的 include 列表里没有任何 `app/` 或 `imgui`。
2. **界面与内核之间只有三条通道：**`submit()`（写）、`snapshot()`（读结构）、`events().tail()`（读日志）。界面从不直接访问 `queue_`、`workers_`、`mutex_`。
3. **越往下越"纯"：**组件层（队列 / 线程壳 / 事件环 / 指标 / 假负载）互不认识；基础层（`Task`、`Priority`、`PoolConfig`）是纯数据 + 纯函数，没有任何锁。

2.2 运行流程图（三个线程角色 + 一次任务的一生）

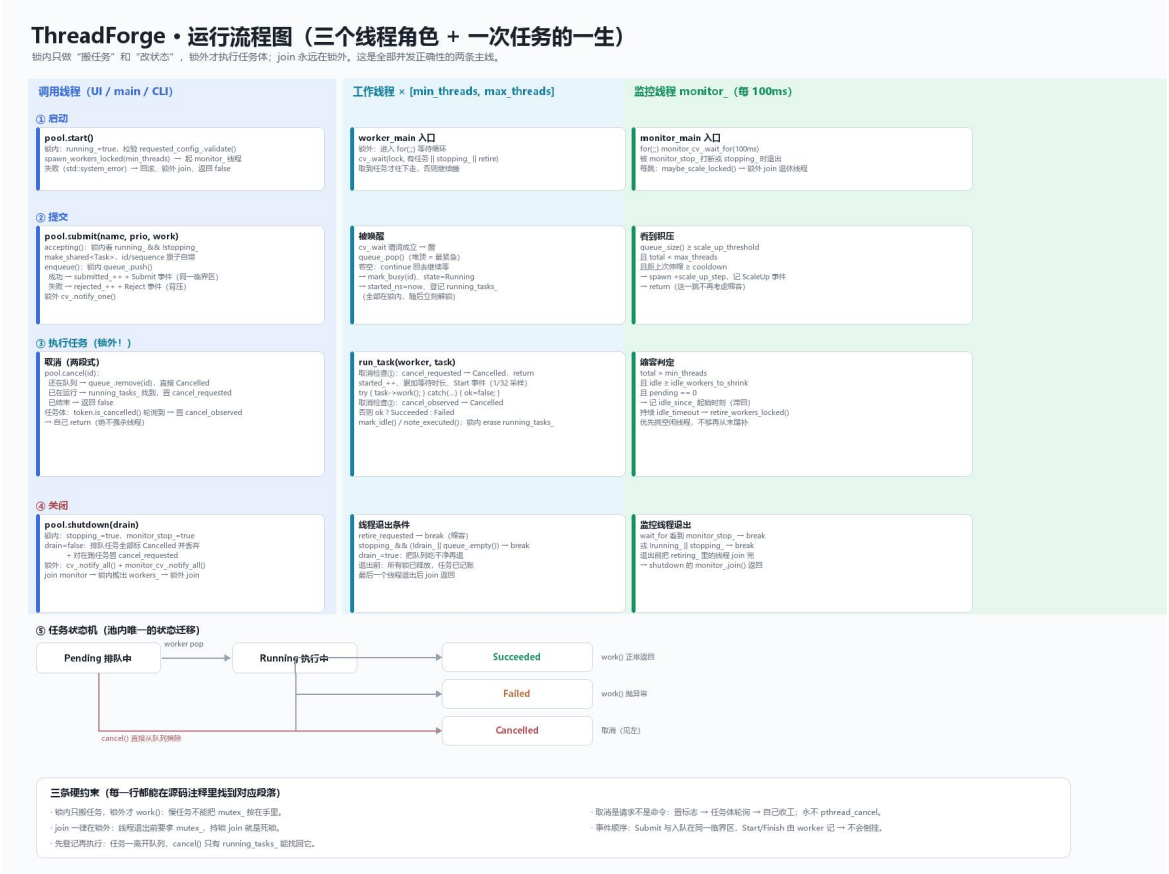


图 2　运行流程图

这张图是**读 `src/pool.cpp` 的地图**：

- 左列是**调用线程**（UI 主线程或 CLI 主线程）：`start / submit / cancel / shutdown` 都由它发起。
- 中列是**工作线程**：它们只做一件事——`pop` 一个任务，锁外执行，回来记一笔。
- 右列是**监控线程**：每 100ms 醒一次，决定扩容、缩容、还是重建优先级堆。
- 最下面是任务状态机：`Pending → Running → Succeeded / Failed / Cancelled`，以及两条取消路径。

图里标红的两条主线，是整个项目并发正确性的全部：

1. **锁内只搬任务，锁外才执行**——`worker_main()` 在锁内 `pop` + 标记 + 登记，然后**立刻解锁**，`run_task()` 在锁外跑。
2. **join 一律在锁外**——线程退出前要拿 `mutex_`，任何"持锁 join"都是死锁。

3. 构建、运行与测试

3.1 构建

```
# 在仓库根目录
cmake -S . -B build/debug -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug
```

产物统一落在 `build/debug/bin/`（`CMakeLists.txt` 里设置了 `CMAKE_RUNTIME_OUTPUT_DIRECTORY`）。

仓库里还有 `CMakePresets.json`（`mingw-debug` / `mingw-release` 两个预设）和 `.vscode/tasks.json`，用 VS Code 时可以直接按任务构建。

> **中文目录的坑**：项目路径里带中文时，MinGW 的 `__FILE__` 会变成非 ASCII 字面量，`assert()` 展开后编译器会报 `converting to execution character set: Illegal byte sequence`。
>
> `CMakeLists.txt` 用 `-ffile-prefix-map=${CMAKE_SOURCE_DIR}=.` 把源码根映射成 `.` 解决了它，顺带让调试器里的路径更短。

3.2 三个可执行文件

```
# ① 图形仪表盘
build\debug\bin\threadforge_app.exe
build\debug\bin\threadforge_app.exe --demo     # 启动即开持续压测，适合截图

# ② 无界面压测（先跑这个，再开界面）
build\debug\bin\threadforge_cli.exe --tasks 800 --threads 2 --max 12 --workload cpu
build\debug\bin\threadforge_cli.exe --tasks 500 --workload io --batch 10 --pause 5

# ③ 单元测试
build\debug\bin\tp_tests.exe
ctest --test-dir build/debug --output-on-failure
```

CLI 的参数：

参数	默认	含义
`--tasks N`	300	总共提交多少个任务
`--threads N`	2	初始（= 最少）线程数
`--max N`	8	扩容上限
`--batch N`	25	每批提交多少个（批间停顿让监控线程看见积压）
`--pause MS`	20	批间停顿毫秒数
`--workload KIND`	`mixed`	`cpu` / `io` / `mixed`
`--help`	—	打印用法

非法参数一律**回落默认值**而不是报错退出（例如 `--tasks -5` 会退回 300，

`--batch abc` 会退回 25）。这是 `parse_size()` 的职责，见 `apps/cli_main.cpp:66`。

3.3 界面怎么用

1. 右上角 **启动线程池**（或直接 `--demo` 启动）。
2. 左栏拖滑块：线程范围、扩容阈值、冷却、缩容门槛、队列容量、老化速度——**拖动即生效**，线程池运行中也能改。
3. **提交一批任务**，或勾上 **持续压测** 按速率自动提交。
4. 中栏看指标卡与线程网格（橙 = 忙，灰 = 闲，红 = 退休中），

下方曲线看线程数 / 积压 / 平均等待 / 吞吐。

5. 底部左边是**等待队列**（按调度顺序，前 64 条），右边是**事件日志**（最近 200 条）。
6. 右上角：**优雅关闭**（跑完队列里的任务）/ **立即停止**（丢弃排队任务）/ 清零统计 / 清空日志 / 重置配置。

4. 目录结构与文件清单

```
多线程池/
├── CMakeLists.txt              构建脚本：4 个编译目标 + 2 个便捷目标
├── CMakePresets.json           mingw-debug / mingw-release
├── README.md                   项目简介（功能列表、构建、测试）
│
├── include/threadpool/         ← 内核头文件（装出来的"公共 API"）
│   ├── task.hpp                Task / Priority / TaskState / CancellationToken（纯数据 +
inline）
│   ├── config.hpp              PoolConfig：12 个可调参数 + validate / sanitized
│   ├── priority_queue.hpp      带优先级与老化的任务队列（刻意非线程安全）
│   ├── event_log.hpp           线程安全的事件环形缓冲
│   ├── worker.hpp              一个工作线程的"躯壳"
│   ├── metrics.hpp             WorkerInfo / TaskInfo / PoolSnapshot（纯值快照）
│   ├── simulator.hpp           假负载生成器
│   └── pool.hpp                线程池本体（唯一的"大脑"）
│
├── src/                        ← 内核实现
│   ├── config.cpp              validate 的 13 条规则 + sanitized 的逐项夹紧
│   ├── priority_queue.cpp      二叉堆 + 老化 + 装饰-选择式 snapshot
│   ├── event_log.cpp           环形缓冲读写 + 覆盖计数
│   ├── worker.cpp              launch / join / mark_busy / note_executed
│   ├── simulator.cpp           Cpu 忙等 / Io sleep / 混合
│   └── pool.cpp                submit / worker_main / monitor_main / shutdown（738 行）
│
├── app/                        ← 界面层（只依赖 tp_core 的公共 API）
│   ├── main.cpp                Win32 窗口 + D3D11 设备 + ImGui 主循环（287 行）
│   ├── theme.hpp / theme.cpp   配色 Palette + 中文字体烘焙 + DPI 缩放（240 行）
│   └── dashboard.hpp / .cpp    仪表盘：五块面板 + 四条曲线 + 两张表（748 行）
│
├── apps/
│   └── cli_main.cpp            无界面压测程序（285 行）
│
├── tests/
│   ├── test_framework.hpp      手写测试框架：TF_TEST / TF_CHECK / run_all（113 行）
│   ├── test_main.cpp           入口：return tftest::run_all();
│   ├── test_priority_queue.cpp 6 个队列用例
│   └── test_pool.cpp           16 个线程池用例（含 3 个取消用例 + 3 个工程性用例）
│
└── third_party/imgui/          Dear ImGui 1.91.9b（11 个核心文件 + 4 个后端文件，MIT）
```

文件	行数	一句话职责	有锁？
include/threadpool/task.hpp	238	任务的数据模型 + 取消令牌	否
include/threadpool/config.hpp	47	12 个可调参数	否
include/threadpool/priority_queue.hpp	79	优先级队列接口 + 老化策略	否（刻意）
include/threadpool/event_log.hpp	71	事件环形缓冲接口	内部有锁
include/threadpool/worker.hpp	64	工作线程壳	原子量，无锁
include/threadpool/metrics.hpp	70	三种纯值结构	否
include/threadpool/simulator.hpp	45	假负载发生器	否（提交侧单线程用）

文件	行数	一句话职责	有锁？
include/threadpool/pool.hpp	170	线程池 API + 全部成员	声明 mutex_
src/config.cpp	77	校验 + 夹紧	否
src/priority_queue.cpp	158	堆算法 + 老化 + 快照	否
src/event_log.cpp	93	环形缓冲	是
src/worker.cpp	59	线程壳实现	原子量
src/simulator.cpp	84	假负载实现	否
src/pool.cpp	738	线程池全部逻辑	**是（唯一）**
app/main.cpp	287	Win32 + D3D11 + 主循环	否
app/theme.hpp / theme.cpp	53 / 187	配色 + 字体 + DPI	否
app/dashboard.hpp / .cpp	89 / 673	仪表盘绘制与数据流	否
apps/cli_main.cpp	259	无界面压测	否
tests/test_framework.hpp	113	测试框架	否
tests/test_pool.cpp	544	16 个池用例	—
tests/test_priority_queue.cpp	136	6 个队列用例	—
CMakeLists.txt	118	4 个目标 + 严格警告 + 测试超时	—

5. 逐文件说明

下面按「依赖方向」从底往上讲：先是最纯的数据（`task.hpp`、`config`），

再到互不认识的组件，然后是线程池本体，最后是界面、CLI、测试和构建脚本。

5.1 `include/threadpool/task.hpp`（238 行）— 任务的数据模型

职责：定义"一个任务长什么样"。这里没有任何线程池逻辑，只有数据结构和几个纯函数。

全部函数都是 `inline`（原来在 `src/task.cpp`，后来合并进头文件，因为每个函数只有几行）。

为什么单独拆出来：任务的生命周期状态会被工作线程写、被界面线程读，

所以状态字段必须是原子的——这是后面所有并发逻辑的地基。

5.1.1 时间工具（第 32–41 行）

函数	签名	作用
`to_ns`	`inline long long to_ns(TimePoint t)`	时间点 → 纳秒计数（`long long`）
`from_ns`	`inline TimePoint from_ns(long long ns)`	纳秒计数 → 时间点
`ms_between`	`inline double ms_between(TimePoint a, TimePoint b)`	两个时间点相差多少毫秒（`double`，带小数）

为什么要有 `to_ns/from_ns`：`std::atomic<std::chrono::time_point>` 不存在，

但 `steady_clock` 的 duration 是整数，可以无损塞进 `long long` 再放进 `atomic<long long>`。

所以任务里的 `started_ns/finished_ns` 都存纳秒计数。

5.1.2 优先级（第 46–90 行）

```
enum class Priority : int { Low = 0, Normal = 1, High = 2, Critical = 3 };
constexpr int kPriorityLevels = 4;
constexpr Priority kPriorityList[kPriorityLevels] = {Low, Normal, High, Critical};
```

函数	签名	行为
`to_string`	`inline const char* to_string(Priority p)`	返回 `"Low"/"Normal"/"High"/"Critical"`，用于日志（英文避免控制台乱码）
`priority_label_zh`	`inline const char* priority_label_zh(Priority p)`	返回 `"低"/"普通"/"高"/"紧急"`，用于界面
`priority_slot`	`inline int priority_slot(Priority p)`	`Priority → 0..3` 的下标；**越界时夹到 Normal**（不越界读表）
`priority_from_slot`	`inline Priority priority_from_slot(int slot)`	下标 → `Priority`；**越界夹紧到两端**（`-7 → Low`，`99 → Critical`）

> 这四个函数曾是"Day 1 空壳"（`return "";` / `return -1;`），空壳期间界面的"优先级"整列是空白、下拉框选"高/紧急"都退化成 Normal——这是"接口约定没实现"最典型的症状。

5.1.3 任务状态机（第 105–133 行）

```
enum class TaskState {
    Pending,    // 在队列里排队
    Running,    // 已被某个 Worker 取走
    Succeeded,  // 正常结束
    Failed,     // 抛异常结束
    Cancelled,  // 未执行完就被取消
};
```

函数	行为
`to_string(TaskState)`	`"Pending"/"Running"/"Succeeded"/"Failed"/"Cancelled"`
`task_state_label_zh(TaskState)`	`"排队"/"执行中"/"成功"/"失败"/"已取消"`

关键约定：`is_finished()` 用 `state >= TaskState::Succeeded` 判断，

这依赖**枚举顺序**——三个终态必须排在 `Running` 之后。想加状态（比如 `TimedOut`）时，

只能加在 `Cancelled` 后面，否则 `is_finished()` 会误判。

5.1.4 `struct Task`（第 146–197 行）

成员	类型	谁写	谁读	说明
`id`	`TaskId`	提交线程	所有地方	全局唯一，单调递增
`name`	`std::string`	提交线程	界面、日志	如 `"Mixed-1957"`
`priority`	`Priority`	提交线程	队列比较器、界面	静态优先级
`sequence`	`std::uint64_t`	提交线程	队列比较器	同优先级时保证 FIFO 且可复现
`work`	`std::function<void()>`	提交线程	工作线程	真正干活的可调用对象
`submitted_at`	`TimePoint`	提交线程	界面、统计	入队时刻，之后只读
`state`	`std::atomic<TaskState>`	工作线程	界面、`cancel()`	生命周期状态
`cancel_requested`	`std::atomic<bool>`	`cancel()` / `shutdown(false)`	工作线程	"有人请求取消"
`cancel_observed`	`std::atomic<bool>`	`CancellationToken::is_cancelled()`	`run_task()`	"任务体**真的**看到了取消"
`started_ns`	`std::atomic<long long>`	工作线程	界面、统计	开始执行的纳秒时刻，0 = 未开始
`finished_ns`	`std::atomic<long long>`	工作线程	界面、统计	结束的纳秒时刻，0 = 未结束

成员函数：

函数	签名	行为
`waited_ms`	`double waited_ms(TimePoint now) const`	Pending → `now - submitted_at`；已开始 → 定格在 `started_ns - submitted_at`；未开始也无时间戳 → 0
`run_ms`	`double run_ms(TimePoint now) const`	未开始 0；已结束 `finished - started`；执行中 `now - started`
`is_pending/is_running/is_finished`	`bool ... () const`	读 `state`（`is_finished` 用 `>= Succeeded`）
`cancel_requested_flag`	`bool cancel_requested_flag() const`	读取消标志（调试/测试用）

拷贝与移动全部 `= delete`：`Task` 里有 `std::atomic` 成员，天然不可拷贝；

所有传递一律用 `TaskPtr = std::shared_ptr<Task>`。

5.1.5 `class CancellationToken`（第 209–236 行）

交给任务体的"一只耳朵"：任务在长循环里轮询它，发现被取消就自己 `return`。

```
class CancellationToken {
public:
    CancellationToken() = default;
    explicit CancellationToken(std::weak_ptr<Task> task);
    bool valid() const noexcept;          // !task_.expired()
    bool is_cancelled() const noexcept;   // 命中时顺带置 cancel_observed
    TaskId id() const noexcept;           // 任务已释放时返回 0
private:
    std::weak_ptr<Task> task_;
};
```

函数	关键逻辑
构造	一般由 `ThreadPool::submit` 的重载构造；**构造函数是 public**，因为包装任务体的 lambda 在成员函数里不继承外层类的友元权限
`valid()`	`task_.lock()` 失败（任务已结束并释放）→ false，此时"取消"已无意义
`is_cancelled()`	`lock()` 成功且 `cancel_requested` 为真 → **置 `cancel_observed = true`** 并返回 true
`id()`	锁住则返回 `task->id`，否则 0

为什么用 `weak_ptr` 而不是 `shared_ptr`：token 常常被任务体捕获进 `Task::work` 里，

若持强引用就形成 `Task → work → 闭包 → token → Task` 的环，任务结束后永远不释放。

`weak_ptr` 只在任务还活着时有意义，正好够用。

为什么 `cancel_observed` 要单独记一笔：池子要区分两种"取消"——

任务体看到信号主动收工（记 `Cancelled`）、任务体压根没理把活干完了（仍记 `Succeeded`）。

把后者谎报成 `Cancelled` 比不取消更难查。

5.2 `include/threadpool/config.hpp`（47 行）+ `src/config.cpp`（77 行）— 配置与校验

职责：把"魔法数字"集中成 12 个可调参数，并提供两个动作：

`validate()`（体检，返回错误清单）和 `sanitized()`（夹紧，把非法值压回合法范围）。

为什么 `sanitized()` 存在：界面滑块和配置文件都可能给出非法组合（比如 `max < min`）。

线程池不能因为用户拖滑块就崩，所以"先夹紧再生效"；但同时要把**原始请求**记下来，

否则下次 `start()` 永远以为配置是合法的（见 5.4 的两份配置）。

5.2.1 12 个参数

字段	默认值	含义	影响谁
`min_threads`	4	启动即创建、空闲也不销毁	`start()`、缩容下限
`max_threads`	12	动态扩容上限	扩容、`retire_workers_locked`
`queue_capacity`	2048	队列满即拒绝（背压）	`queue_.push`、CLI 里设成 4096
`scale_up_threshold`	8	积压达到多少就认为"忙不过来"	`maybe_scale_locked`
`scale_up_step`	2	每次扩容加几个线程	同上
`scale_cooldown_seconds`	0.3	两次伸缩的最小间隔（防抖动）	同上
`scale_down_step`	1	每次缩容回收几个	同上
`idle_workers_to_shrink`	3	空闲线程数达到多少才允许缩容	同上
`idle_timeout_seconds`	2.0	持续空闲多久才回收（滞回）	同上
`aging_enabled`	true	是否启用优先级老化	`effective_level`、`rebuild`
`aging_levels_per_second`	0.2	每等 1 秒，有效优先级涨 0.2 级	同上
`aging_max_bonus_levels`	3	老化加成上限（级）	同上

5.2.2 `validate()`：13 条规则（`src/config.cpp:12-55`）

不抛异常，只返回中文错误清单——配置来自界面输入，用户改到一半时程序不该崩。

#	规则	错误文案（节选）

#	规则	错误文案（节选）
1	`min_threads >= 1`	最小线程数必须 ≥ 1
2	`max_threads >= min_threads`	最大线程数不能小于最小线程数
3	`max_threads <= 1024`	最大线程数建议 ≤ 1024
4	`queue_capacity >= 1`	队列容量必须 ≥ 1
5	`scale_up_threshold >= 1`	扩容阈值必须 ≥ 1
6	`scale_up_threshold <= queue_capacity`	扩容阈值不能大于队列容量（否则永远等不到扩容）
7	`scale_up_step >= 1`	每次扩容步长必须 ≥ 1
8	`scale_down_step >= 1`	每次缩容步长必须 ≥ 1
9	`scale_cooldown_seconds >= 0`	扩容冷却时间不能为负
10	`idle_timeout_seconds >= 0`	空闲回收时间不能为负
11	`aging_levels_per_second >= 0`	老化速度不能为负
12	`aging_max_bonus_levels >= 0`	老化加成上限不能为负
13	`idle_workers_to_shrink >= 1`	缩容所需的空闲线程数必须 ≥ 1

5.2.3 `sanitized()`：逐项夹紧（`src/config.cpp:57-75`）

字段	夹到
`min_threads`	`[1, 1024]`
`max_threads`	`[min_threads, 1024]`（先保证 ≥ min，再封顶）
`queue_capacity`	`[1, 1<<20]`
`scale_up_step` / `scale_down_step`	`[1, max_threads]`
`scale_up_threshold`	`[1, queue_capacity]`
`idle_workers_to_shrink`	`[1, max_threads]`
`scale_cooldown_seconds`	`[0, 60]`
`idle_timeout_seconds`	`[0, 600]`
`aging_levels_per_second`	`[0, 100]`
`aging_max_bonus_levels`	`[0, 100]`

这里有个必须知道的推论：`sanitized()` 的每个边界都恰好满足 `validate()` 的全部规则，所以 `validate(sanitized(x))` **永远是空**。也就是说：

- 拿 `config_`（夹紧后的）去 `validate()`，那条失败分支是**死代码**；
- 想真正实现"配置非法就拒绝启动"，必须校验**原始请求**，于是线程池里存了两份配置（`config_` 与 `requested_config_`，见 5.4.2）。

5.2.4 两个函数的调用场景

场景	调用哪个	结果
`ThreadPool` 构造	`sanitized()`	内部永远持有合法值
`start()`	`requested_config_.validate()`	非空 → 记 Warn 事件并返回 false
`update_config()`	`sanitized()` + `validate()`	夹紧生效，同时把原始值记进 `requested_config_`，非法则记一

场景	调用哪个	结果
		条 Warn
界面滑块	联动区间 + `AlwaysClamp`	从源头滑不出非法值，`sanitized()` 基本无事可做
CLI	不校验	参数解析时就夹紧了（`min ≥ 1`、`max ≥ min`）

5.3 `include/threadpool/priority_queue.hpp`（79 行）+ `src/priority_queue.cpp`（158 行）— 优先级队列

职责：回答两个纯粹的问题——**谁先走**（优先级 + 老化 + FIFO）、**满了怎么办**（拒绝，形成背压）。

最重要的设计决策：这个类刻意不做线程安全。它不知道 `mutex_` 的存在，

加锁责任完全在 `ThreadPool` 手里。好处有两个：

1. 它可以用单元测试直接测（`test_priority_queue.cpp` 的 6 个用例**一个线程都没起**）；
2. 队列内部的算法（堆操作、老化、排序）可以随便改，不用考虑锁的层级。

5.3.1 `struct AgingPolicy`

```
struct AgingPolicy {
    bool enabled = true;
    double levels_per_second = 0.2;  // 每等待 1 秒，有效优先级涨 0.2 级
    int max_bonus_levels = 3;        // 加成上限
};
```

解决什么问题：如果只用静态优先级，只要持续有 High 任务进来，Low 任务就永远排在后面——**饿死**。老化让"等得越久越接近队首"：Low 等 5 秒 = 0+1.0，等 15 秒 = 0+3.0（封顶），

高优先级任务一直轮转的话，Low 最多等 `max_bonus_levels / levels_per_second` 秒就会被提上来。

5.3.2 每个函数

函数	签名	行为与复杂度
构造	`explicit PriorityTaskQueue(std::size_t capacity = 2048, AgingPolicy aging = {})`	`capacity_ = capacity == 0 ? 1 : capacity`；`heap_.reserve(64)` 避免启动期反复扩容
`effective_level`	`double effective_level(const Task& task, TimePoint now) const`	关老化 → 直接返回静态等级；否则 `base + min(等待秒数 × levels_per_second, max_bonus_levels)`，负数归零。**每次调用都做一次 `duration<double>` 除法**，这是后面性能优化针对的热点

函数	签名	行为与复杂度
less_urgent	bool less_urgent(const TaskPtr& a, const TaskPtr& b, TimePoint now) const	堆比较器：等级不同 → 等级低的"更小"（沉到底）；等级相同 → a->sequence > b->sequence，即**先提交的排前面**。用序号而不是指针，保证顺序可复现、能写断言
push	bool push(TaskPtr task)	空指针 false；size >= capacity_ → ++rejected_total_ 并返回 false（背压）；否则 heap_.push_back + std::push_heap，O(log n)
pop	TaskPtr pop()	空 → nullptr；std::pop_heap 把堆顶换到末尾再 pop_back，O(log n)
remove	TaskPtr remove(TaskId id)	线性扫 O(n) 找 id（取消是低频操作）；末尾元素补位 + rebuild() O(n)
clear	void clear()	heap_.clear()
rebuild	void rebuild()	size < 2 直接返回；否则用当前时间重算比较器并 std::make_heap，O(n)
snapshot	std::vector<TaskPtr> snapshot(std::size_t max_items) const	**只服务界面**：先给每个任务算一次等级（装饰），再用 std::partial_sort 取前 k 个（选择），只为最终入选的 k 个任务拷贝 shared_ptr
访问器	size/empty/capacity/set_capacity/aging/set_aging/pushed_total/rejected_total	都是平凡的读/写；set_capacity **不做 clamp**，调用方（update_config）负责传合法值

5.3.3 为什么必须有 rebuild()（以及它后来被限流）

堆里存的是**静态下标**，但"谁更紧急"的分数**随时间变化**：一个 Low 任务越等越接近队首。

堆一旦建好，堆序就只对建堆那一刻的分数成立。所以需要一个周期性动作把"当前分数"重新算一遍。

它曾经的问题：监控线程每 100ms 无条件调一次 rebuild()，而 100ms 里老化只走了 0.02 级——纯属白干。现在改成按"攒够 kAgingRebuildLevelStep = 0.1 级"限流：

```
重建间隔 = max(100ms, 0.1 / aging_levels_per_second 秒)
```

默认 0.2 级/秒 ⇒ 0.5 秒重建一次（比原来省 5 倍）；老化速度调快时间隔自动缩短。

另一次被撤回的尝试（注释保留在 src/priority_queue.cpp:100-107）：

把 rebuild() 也改成"装饰-索引-堆化"（和 snapshot() 同一套路）。实测结果是

Debug 下快 1.5 倍、**Release 下反而慢 36%**——-O2 早已把 effective_level 内联成几次乘加，多出来的三份数组在 2048 个任务时压垮了 L1。结论：这里保持最简写法，收益在**调用频率**上。

5.3.4 `snapshot()` 的两次实现（性能教训）

版本	做法	n=2048、k=64 的实测（Debug）
旧	全量拷贝 `heap_` → `std::sort` 全排序 → 事后 `resize(64)`	**1340 µs/次**
新	每任务算一次等级 → `partial_sort` 取前 64 → 只为 64 个拷贝 `shared_ptr`	**104 µs/次（≈12.8×）**

界面 60 帧/秒调用它，旧写法每帧 1.34 ms **且全程持有线程池 `mutex_`**——等于每 16ms 卡住一次调度。

新写法的比较器只比预先算好的 `double`，比较次数从 O(n log n) 降到 O(n log k)。

5.4 `include/threadpool/event_log.hpp`（71 行）+ `src/event_log.cpp`（93 行）— 事件环形缓冲

职责：让界面能回答"刚才发生了什么"。固定 512 条，写满则覆盖最旧的，

内存恒定、写入 O(1)，不会因为程序跑了一整天而吃掉几个 G。

5.4.1 事件种类

枚举	中文标签	谁的什么动作会记它
`Submit`	提交	`enqueue()` 成功入队（**1/32 采样**，可开全量）
`Start`	开始	`run_task()` 开始执行（采样）
`Finish`	完成	任务正常返回（采样）
`Fail`	失败	任务体抛异常（**不采样**，重要事件一条不落）
`Cancel`	取消	排队中被摘除 / 运行中按取消收工 / 非优雅关闭丢弃
`Reject`	拒绝	队列满，提交被拒绝
`ScaleUp` / `ScaleDown`	扩容 / 缩容	`maybe_scale_locked()`
`Info`	信息	启动、关闭、配置已更新、计数器清零
`Warn`	警告	取消失败、配置非法、启动失败回滚

`to_string(EventKind)` 给日志用（英文），`event_label_zh(EventKind)` 给界面用（中文）。

5.4.2 数据结构

```
struct PoolEvent {
    TimePoint at{};          // 发生时刻
    EventKind kind = Info;
    TaskId task = 0;         // 0 表示与具体任务无关
```

```
    std::string text;        // 已经格式化好的整句话
};
```

成员	含义
mutex_	保护 buffer_ / head_ / size_ / dropped_
buffer_	std::vector<PoolEvent>，构造时按容量一次性开好（默认 512）
head_	**下一条要写的位置**（不是最旧的位置）
size_	当前有效条数（未满时递增，满了以后恒等于容量）
total_	std::atomic<uint64_t>：**累计**产生过多少条（清空也不回头）
dropped_	因为写满而覆盖掉的条数（界面用来提示"日志有丢失"）

5.4.3 每个函数

函数	签名	关键逻辑
构造	explicit EventLog(std::size_t capacity = 512)	capacity == 0 ? 1 : capacity，直接 vector(cap) 分配
add	void add(EventKind kind, std::string text, TaskId task = 0)	先在锁外组装 PoolEvent（Clock::now() + move(text)），再进锁：满了 → ++dropped_，没满 → ++size_；写 buffer_[head_]，head_ = (head_+1) % cap，最后 total_++
tail	std::vector<PoolEvent> tail(std::size_t n) const	锁内取 count = min(n, size_)，起点 start = (head_ + cap - count) % cap，按正序拷出（最旧的在前）
clear	void clear()	锁内 head_ = 0; size_ = 0; dropped_ = 0; 并把 total_ 也清零
total	uint64_t total() const	读原子量（不加锁）
dropped	uint64_t dropped() const	锁内读 dropped_

一个曾经的小 bug：clear() 最初只清 head_/size_/dropped_，不清 total_，

于是"清空日志"之后界面上的累计数还在涨，看起来像清空失败。现在两者一起归零。

采样与全量：任务级事件默认按 id & 0x1F == 0 采样（1/32）。

吞吐上万时，日志的加锁与字符串分配本身会变成新瓶颈；失败、取消、拒绝、扩缩容这些

"重要但低频"的事件一条不落。要追某个具体任务时，

ThreadPool::set_task_event_full_trace(true)

（界面左栏也有勾选框）打开全量记录。

5.5 `include/threadpool/worker.hpp`（64 行）+ `src/worker.cpp`（59 行）— 工作线程的"躯壳"

职责：只做三件事——拥有一个 `std::thread`、记录自己现在忙不忙、累计自己干过多少活。

它不知道队列怎么调度，调度逻辑全在 `pool.cpp`。这条职责分界线是读懂 `pool.cpp` 的前提。

5.5.1 成员

成员	类型	谁写	说明
`index_`	`std::size_t`	构造	线程编号（从 1 开始），界面显示 `W1/W2/...`
`thread_`	`std::thread`	`launch()`	真正的线程
`wake_on_retire_`	`std::function<void()>`	`launch()`	析构兜底用的唤醒器，见 5.5.2
`busy_`	`atomic<bool>`	`mark_busy/mark_idle`	是否在跑任务
`current_`	`atomic<TaskId>`	同上	当前任务 id，0 = 空闲
`executed_`	`atomic<uint64_t>`	`note_executed`	累计执行过多少个任务
`busy_us_`	`atomic<int64_t>`	`note_executed`	累计忙碌微秒数（`busy_ms()` 除以 1000 得到毫秒）
`retire_requested`	`atomic<bool>`（public）	`retire_workers_locked`	缩容时置位；工作线程看到就自行退出

5.5.2 每个函数

函数	签名	行为
构造	`explicit Worker(std::size_t index)`	只记编号
析构	`~Worker()`	先 `retire_requested = true`，再调 `wake_on_retire_()`，最后 `join()`
`launch`	`void launch(std::function<void(Worker&)> body, std::function<void()> wake_on_retire = {})`	已经 `joinable()` 就静默返回（防重复启动）；保存 `wake_on_retire_`；`thread_ = std::thread([this, body]{ body(*this); })`
`join`	`void join()`	`joinable()` 才 join
`joinable`	`bool joinable() const`	转发给 `thread_`
`mark_busy`	`void mark_busy(TaskId id)`	先存 `current_` 再存 `busy_`（都是 release），顺序保证**看到 busy=true 时 current_ 一定已经是新任务**
`mark_idle`	`void mark_idle()`	先清 `busy_` 再清 `current_`
`note_executed`	`void note_executed(double run_ms)`	`executed_++; busy_us_ += run_ms * 1000`（负数归零防累计出错）
`reset_stats`	`void reset_stats()`	`executed_` 与 `busy_us_` 归零（界

函数	签名	行为
		面"清零统计"）
busy_ms	double busy_ms() const	busy_us_ / 1000.0

析构为什么必须 notify：线程平时睡在 ThreadPool 的 cv_ 上。只置 retire_requested 而不唤醒，条件变量的谓词根本不会被重新求值，线程照样睡着，join() 就会一直等下去。

wake_on_retire_ 是 ThreadPool 在 spawn_workers_locked 里传进来的 [this]{ cv_.notify_all(); }，

专门给"没人 notify 就析构了"这种兜底路径用。

5.6 include/threadpool/metrics.hpp（70 行）— 体检报告

职责：定义三种**纯值结构**，是界面唯一能看到的数据形态。界面绝不直接摸线程池内部。

结构	字段	说明
WorkerInfo	index / busy / retiring / current_task / executed / busy_ms	一个线程的状态（对应界面上的一个方块）
TaskInfo	id / name / priority / effective_level / waited_ms	队列里一个任务的"视图"（只拷界面需要的字段，不持有 TaskPtr）
PoolSnapshot	running / shutting_down / threads / busy_threads / idle_threads / retiring_threads / queue_size / queue_capacity / submitted / completed / failed / cancelled / rejected / avg_wait_ms / avg_run_ms / uptime_seconds / config / workers / queue_view	一次"拍照"的全部内容

PoolSnapshot 是**值拷贝**：snapshot() 返回之后，界面想怎么读、想存多久都可以，不用再拿锁。这就是"每帧抓一张照片"的写法。

TaskInfo 不持有 TaskPtr 是刻意的：如果持有，界面缓存 200 个任务就等于让 200 个任务迟迟不能释放；而且界面只是显示，不需要任务本体。

5.7 include/threadpool/simulator.hpp（45 行）+ src/simulator.cpp（84 行）— 假负载

职责：造出"看起来很真实"的假任务，让线程池有活干。

关键设计：随机数在**生成任务时**就抽好，任务体只捕获几个普通变量。

于是工作线程执行任务时不需要碰 rng_，天然没有数据竞争——

`WorkloadFactory` 本身因此是**非线程安全**的，只在提交任务的那一侧用。

5.7.1 三种负载

种类	实现	观察什么
`Cpu`	FNV-1a 哈希忙等到 deadline（2~19ms × intensity）	CPU 打满时的扩容行为
`Io`	`sleep_for(2~31ms × intensity)`	线程数远超核数也能提吞吐（sleep 不占 CPU）
`Mixed`	55% CPU / 45% Io	混合场景，扩缩容曲线最好看

CPU 任务用"跑到截止时间"而不是固定迭代次数：换机器、换优化等级，耗时依然大致可控。

循环里用 `volatile std::uint64_t sink = h;` 写一次，防止编译器把整段计算当死代码优化掉。

5.7.2 每个函数

函数	签名	行为
构造	`explicit WorkloadFactory(std::uint32_t seed = 20251029u)`	初始化 `std::mt19937`
`reseed`	`void reseed(std::uint32_t seed)`	重设种子（想让两次运行完全一致时用）
`make`	`std::function<void()> make(WorkloadKind kind, double intensity = 1.0)`	非法/NaN 强度 → 1.0；`Mixed` 先掷骰子决定这次是 CPU 还是 Io；返回**自包含**的 lambda
`random_priority`	`Priority random_priority()`	金字塔分布：Low 30% / Normal 45% / High 20% / Critical 5%（否则优先级队列和没有优先级没区别）
`to_string`	`const char* to_string(WorkloadKind)`	`"Cpu"/"Io"/"Mixed"`（日志）
`workload_label_zh`	`const char* workload_label_zh(WorkloadKind)`	`"计算密集"/"IO 等待"/"混合"`（界面）

5.8 `include/threadpool/pool.hpp`（170 行）— 线程池的对外契约

职责：声明线程池的全部公开动作、私有实现函数、以及所有共享状态。

读这个头文件要带着一个问题：**这个变量由谁写、被谁读、要不要加锁**。

5.8.1 两个界面共用常量（第 47–48 行）

```
inline constexpr std::size_t kQueueViewLimit = 64;   // 界面一次最多显示多少条排队任务
inline constexpr std::size_t kEventViewLimit = 200;  // 界面一次最多显示多少条事件
```

放在内核头文件里是为了让界面和内核共用同一个上限，而不是在 `dashboard.cpp` 里再抄一遍魔数

（之前就是抄的，改一处忘一处）。

5.8.2 公开 API

方法	签名	语义
构造	`explicit ThreadPool(PoolConfig config = PoolConfig{})`	把配置夹紧（`sanitized()`）后存成 `config_`，原始值存成 `requested_config_`；队列按 `config_` 建
析构	`~ThreadPool()`	兜底 `shutdown(true)`，避免 `std::thread` 析构时 `std::terminate`
`start`	`bool start()`	建 `min_threads` 个工作线程 + 起监控线程；**校验的是原始请求配置**，非法返回 false
`shutdown`	`void shutdown(bool drain = true)`	`drain=true` 跑完队列再退；`false` 丢弃排队任务后立刻退
`running`	`bool running() const`	`running_ && !stopping_`（锁内读）
`submit`	`std::optional<TaskId> submit(std::string name, Priority priority, std::function<void()> work)`	成功返回任务 id；队列满 / 未启动 / 正在关闭返回 `nullopt`
`submit`（令牌重载）	`std::optional<TaskId> submit(std::string name, Priority priority, std::function<void(const CancellationToken&)> work)`	同上，但任务体额外拿到取消令牌
`cancel`	`bool cancel(TaskId id)`	排队中 → 摘除并返回 true；运行中 → 置标志返回 true（线程不会被打断）；已结束/不存在 → false
`snapshot`	`PoolSnapshot snapshot() const`	锁内拍一张值快照，界面每帧调用
`update_config`	`void update_config(const PoolConfig& config)`	热更新：夹紧生效、记住原始请求、按需增减线程、发"配置已更新"事件
`events`	`const EventLog& events() const`	只读访问事件日志
`clear_events`	`void clear_events()`	转发 `events_.clear()`
`reset_counters`	`void reset_counters()`	所有计数器与 worker 统计一起归零（锁内）
`set_task_event_full_trace`	`void set_task_event_full_trace(bool enabled)`	关掉 1/32 采样，任务级事件全量记录（写原子量，无锁）
`task_event_full_trace`	`bool task_event_full_trace() const`	读该开关

5.8.3 私有函数（按调用方向分组）

组	函数	说明
工作线程	`void worker_main(Worker& worker)`	工作线程主循环（第 5 课重点）
	`void run_task(Worker& worker, const TaskPtr& task)`	执行任务体 + 记账（锁外）
提交路径	`bool accepting() const`	锁内判断"还收不收任务"
	`bool enqueue(const TaskPtr& task)`	锁内入队，失败记背压事件
	`bool should_log_task(TaskId id) const`	任务级事件要不要落盘（采样开关）
监控线程	`void monitor_main()`	每 100ms 巡检（第 6、7 课重点）
	`void maybe_scale_locked()`	🔒 扩容 / 缩容 / 老化重建
	`std::chrono::milliseconds aging_rebuild_interval() const`	老化重建的限流间隔
	`std::size_t busy_count_locked() const`	🔒 数忙线程
	`void spawn_workers_locked(std::size_t count)`	🔒 建线程并启动
	`void retire_workers_locked(std::size_t count)`	🔒 标记退休、挪进 `retiring_`
	`std::vector<std::unique_ptr<Worker>> take_retiring()`	🔒 取走待 join 的线程（join 在锁外做）

5.8.4 状态：哪些受 `mutex_` 保护，哪些是原子的

受 `mutex_` 保护的（必须持锁访问）

成员	类型	作用	关键点
`mutex_`	`mutable std::mutex`	**唯一的锁**	队列、线程表、运行标志、计数器的"切换"都靠它
`cv_`	`condition_variable`	任务到达 / 关闭 / 退休唤醒	工作线程睡在它上面
`config_`	`PoolConfig`	**真正生效的配置**（永远合法）	由构造与 `update_config` 写入
`requested_config_`	`PoolConfig`	调用方最后一次"原样"给的配置	`start()` 拿它做校验；不夹紧
`queue_`	`PriorityTaskQueue`	待执行任务	刻意无锁的容器，靠 `mutex_` 保护
`workers_`	`vector<unique_ptr<Worker>>`	在岗线程	缩容时部分元素被移走
`retiring_`	`vector<unique_ptr<Worker>>`	已标记退休、等待 join	只有锁内搬动，join 在锁外
`running_tasks_`	`unordered_map<TaskId, TaskPtr>`	正在执行的任务	**`cancel()` 找回"已出队"**

成员	类型	作用	关键点
			任务的唯一途径
`running_`	`bool`	是否已启动	`start()` 置 true，`shutdown()` 末尾置 false
`stopping_`	`bool`	是否正在关闭	关闭期间不再收任务
`drain_`	`bool`	优雅关闭	`true` 时要把队列吃干净
`started_at_`	`TimePoint`	启动时刻	用来算"运行秒数"
`last_scale_at_`	`TimePoint`	上次伸缩时刻	冷却判定
`last_aging_rebuild_`	`TimePoint`	上次老化重建时刻	限流判定
`idle_since_`	`TimePoint`	空闲达标起始时刻	缩容滞回（中途不满足就清零重来）
`next_worker_index_`	`std::size_t`	下一个线程编号	从 1 开始，只增不减（编号不复用，界面好看）

监控线程自己的三个成员

成员	说明
`monitor_`	`std::thread`，`start()` 里创建
`monitor_cv_`	监控线程睡在它上面（用 `wait_for` 带超时）
`monitor_stop_`	`bool`，`shutdown()` 置 true 让监控线程退出

无锁计数器（`std::atomic`，工作线程随手累加）

成员	类型	含义
`next_id_`	`atomic<TaskId>`	下一个任务 id（从 1 开始）
`next_seq_`	`atomic<uint64_t>`	下一个提交序号（同优先级 FIFO 用）
`submitted_`	`atomic<uint64_t>`	成功入队的任务数
`started_`	`atomic<uint64_t>`	真正开始执行过的任务数（算平均等待用**分母**）
`completed_`	`atomic<uint64_t>`	正常完成
`failed_`	`atomic<uint64_t>`	抛异常
`cancelled_`	`atomic<uint64_t>`	被取消（排队中摘除 + 运行中收工）
`rejected_`	`atomic<uint64_t>`	因队列满被拒绝
`wait_ns_total_`	`atomic<uint64_t>`	累计等待纳秒（算平均等待的**分子**）
`run_ns_total_`	`atomic<uint64_t>`	累计执行纳秒（算平均执行）
`full_trace_`	`atomic<bool>`	任务级事件是否全量记录

为什么这些用原子而不是锁：工作线程每执行一个任务就要更新 3~4 个计数器，

如果每次都抢 `mutex_`，锁竞争会变成吞吐瓶颈。计数器之间**不需要彼此一致**（快照偶尔偏差一点无所谓），

所以用 `relaxed` 原子累加就够了。

5.9 `src/pool.cpp`（738 行）— 线程池本体

阅读顺序建议： `submit → worker_main → run_task → monitor_main → maybe_scale_locked → shutdown`。

三个最容易写错的地方在源码里用【易错点】标出来了。

5.9.1 文件内的三个常量（第 17–28 行）

常量	值	为什么是这个值
`kMonitorTick`	`100ms`	"反应够快"与"CPU 开销够小"的折中
`kTaskEventSampleMask`	`0x1F`	任务级事件按 1/32 采样（id 低 5 位为 0 才记）
`kAgingRebuildLevelStep`	`0.1` 级	老化攒够 0.1 级重建一次堆就够，见 5.9.13

还有一个小工具 `ns_between(TimePoint a, TimePoint b)`：返回纳秒差（`b <= a` 时返回 0），给"累计等待/执行时长"和"运行秒数"用。

5.9.2 构造与析构（第 41–56 行）

```
ThreadPool::ThreadPool(PoolConfig config)
    : config_(config.sanitized()),        // 生效值：永远合法
      requested_config_(config),          // 原始值：start() 用它校验
      queue_(config_.queue_capacity,      // 队列必须和生效值一致
             AgingPolicy{config_.aging_enabled,
                         config_.aging_levels_per_second,
                         config_.aging_max_bonus_levels}) {}
```

初始化列表的顺序有讲究： `queue_` 的构造要读 `config_`，所以 `config_` 必须声明在 `queue_` 前面（`pool.hpp` 里成员的声明顺序就是按这个排的）。**改成员顺序前先看这一行。**

`~ThreadPool()` 只有一句 `shutdown(true)`：使用者忘了关也不会让 `std::thread` 在析构时报 `terminate called without an active exception`。

5.9.3 `start()`（第 61–124 行）

```
① 锁内：running_ 已为真 → return false
② 锁内：requested_config_.validate() 非空 → 逐条记 Warn 事件，return false
③ 锁内：置 running_=true、stopping_=false、drain_=true、started_at_/last_scale_at_/
        last_aging_rebuild_=now、idle_since_={}、monitor_stop_=false
④ 锁内 try：spawn_workers_locked(min_threads) + monitor_ = std::thread(monitor_main)
   catch(std::system_error)：running_=false、stopping_=false、monitor_stop_=true、
        给已建线程置 retire、搬进 rollback_join、workers_.clear()、记 Warn
⑤ 锁外：失败的路径先 cv_.notify_all()，再逐个 join（锁内 join 会死锁）
⑥ 锁外：记 Info「线程池启动：初始 N 线程，上限 M」
```

三个要点：

1. **校验原始配置**：`config_` 已经被 `sanitized()`，拿它 `validate()` 永远通过，

那条失败分支会成为死代码。所以留了 `requested_config_`。

2. **`monitor_stop_` 每次都重置**：否则第二次 `start()` 时监控线程会立刻退出。

3. **异常安全**：`std::thread` 构造可能抛 `std::system_error`（句柄耗尽等）。

此时 `running_` 已是 true，若让异常逃出去，调用方拿到的是"自称在跑、其实没监控线程"的坏池子，

而 `start()` 声明返回 bool 本就不该抛。所以自己吞掉并回滚：线程标退休 → 锁外 notify → join。

5.9.4 `shutdown(bool drain)`（第 129–184 行）

```
① 锁内：!running_ → return
② 锁内：stopping_=true、drain_=drain、monitor_stop_=true
③ 锁内（仅 !drain）：queue_.pop() 循环把排队任务全部标 Cancelled、cancelled_++、
   并对 running_tasks_ 里每个任务置 cancel_requested（愿意配合的立刻收工，
   不配合的照旧跑完，join 继续等 —— 绝不强杀线程）
④ 锁外：cv_.notify_all() + monitor_cv_.notify_all() → monitor_.join()
⑤ 锁内：把 retiring_ 与 workers_ 全部 move 进 to_join，清空两个容器
⑥ 锁外：逐个 join（【易错点】持锁 join = 死锁）
⑦ 锁内：running_=false、stopping_=false
⑧ 锁外：丢弃数 > 0 记 Cancel 事件；最后记 Info「线程池已关闭」
```

为什么 join 必须在锁外：工作线程退出前要拿 `mutex_` 看一眼状态、`running_tasks_.erase()` 也要拿锁。主线程如果握着锁等它退出，就是经典的"A 持锁等 B，B 等锁"。

5.9.5 `running()` / `accepting()` / `should_log_task()`（第 186–204 行）

函数	实现	说明
`running()`	锁内 `return running_ && !stopping_;`	关闭中也算"没在跑"
`accepting()`	锁内 `return running_ && !stopping_;`	与 `running()` 同义，但语义是"还收不收任务"
`should_log_task(id)`	`full_trace_` 为真 → true；否则 `(id & 0x1F) == 0`	只读一个原子量，**在锁内调用也安全**

5.9.6 `enqueue()`（第 206–232 行）

```
bool ThreadPool::enqueue(const TaskPtr& task) {
    bool pushed = false;
    {
        std::lock_guard lock(mutex_);
        if (!running_ || stopping_) return false;   // 装箱期间池子被关了
        pushed = queue_.push(task);
        if (pushed) {
            submitted_.fetch_add(1, relaxed);
            if (should_log_task(task->id))
                events_.add(EventKind::Submit, "提交任务 " + task->name, task->id);
        }
```

```
    }
    if (!pushed) {
        rejected_.fetch_add(1, relaxed);
        events_.add(EventKind::Reject, "队列已满，提交被拒绝：" + task->name, task->id);
        return false;
    }
    cv_.notify_one();      // 只叫醒一个：一个任务只需要一个线程
    return true;
}
```

为什么 Submit 事件必须在临界区里发：如果先解锁、事后再补日志，

worker 可能已经 `pop` 了它并写下 `Start`/`Finish`，日志里就会出现"先完成、后提交"的倒挂。

在锁内调 `events_.add()` 会**嵌套拿 EventLog 自己的锁**，安全——EventLog 从不回调线程池。

5.9.7 `submit()` 的两个重载（第 237–282 行）

普通版：

```
work 为空 → nullopt
accepting() 为假 → nullopt（先短锁检查，不白白消耗 id 和一次 make_shared）
make_shared<Task>()；id/sequence 用原子自增；填 priority/name/work/submitted_at
enqueue() 失败 → nullopt
返回 task->id
```

带令牌的重载只多一步包装：

```
std::weak_ptr<Task> self = task;
task->work = [body = std::move(work), self]() {
    const TaskPtr alive = self.lock();
    body(CancellationToken(alive));
};
```

为什么用 `weak_ptr` 捕获自己：`Task::work` 是 `Task` 的成员，

闭包再捕获 `shared_ptr<Task>` 就成环（`Task → work → 闭包 → Task`），任务结束后引用计数永远回不到 0。

执行期间 worker 手里一直握着 `shared_ptr`，所以 `lock()` 一定成功。

5.9.8 `cancel()`（第 291–331 行）— 两段式取消

```
锁内：
  task = queue_.remove(id)                    // 第一段：还在排队
  if (task) { was_queued = true; found = true; }
  else {
      it = running_tasks_.find(id)            // 第二段：已经被取走、正在跑
      if (it != end && !it->second->is_finished()) { task = it->second; found = true; }
  }
  if (found) {
      task->cancel_requested.store(true, release);
      if (was_queued) { task->state = Cancelled; cancelled_++; }
  }
锁外：
  !found → 记 Warn「取消失败：任务不存在或已结束」，return false
  was_queued → 记 Cancel「任务已取消（排队中）：…」
  否则 → 记 Cancel「已请求取消（执行中，等任务体收工）：…」
```

三个细节：

1. **`!is_finished()` 这个条件不能省**：worker 收起任务到把它从 `running_tasks_` 摘掉之间有一个很短的窗口，

不判就会把"已经跑完的任务"也报成功取消，返回值不稳定、测试也没法写。

2. **排队中的任务可以直接定终态**（它永远不会被执行）；运行中的任务只能置标志。

3. **绝不强杀线程**：`pthread_cancel` / `TerminateThread` 会跳过锁释放和栈上对象的析构，

池子会以最难查的方式坏掉。

5.9.9 `update_config()`（第 336–386 行）

```
cfg = config.sanitized()
锁内：
  requested_config_ = config;          // 记住原始请求（下次 start() 还要用它校验）
  config_ = cfg;
  problems = config.validate();         // 原始配置非法 → 记一条 Warn（字符串在锁内拼好）
  queue_.set_capacity(cfg.queue_capacity);
  queue_.set_aging(...);
  monitor_alive = monitor_.joinable() && !monitor_stop_;
  if (running_ && !stopping_) {
      while (workers_.size() < cfg.min_threads) spawn_workers_locked(1);   // 下界调大 → 补线程
      if (workers_.size() > cfg.max_threads) retire_workers_locked(超出部分); // 上界调小 → 回
收
  }
锁外：
  if (!monitor_alive) { to_join = take_retiring(); 逐个 join; }
  记 Warn（如果有）与 Info「配置已更新：线程范围 A~B」
```

> 早期版本这里有一句 `if (!running_) return;`，结果"配置已更新"的事件被吞掉了——
>
> 配置明明生效了，界面日志里却没有任何反馈。现在改成**绝不提前 return**。

join 只在监控线程不在时才做：监控线程本来每跳都会回收 `retiring_`，

界面线程如果也跟着 join，就可能被一个正在跑长任务的 worker 卡住（界面卡顿）。

5.9.10 `snapshot()`（第 391–454 行）

```
锁内：
  running/shutting_down/config
  遍历 workers_ → WorkerInfo（index/busy/retiring/current_task/executed/busy_ms），顺带数
busy_threads
  idle_threads = threads - busy_threads
  queue_size / queue_capacity
  五个计数器 + 两个平均值：
      avg_wait_ms = wait_ns_total_ / 1e6 / started_（started_ > 0 时）
      avg_run_ms  = run_ns_total_  / 1e6 / started_（同上）
  uptime_seconds = ns_between(started_at_, now) / 1e9
  队列表：queue_.snapshot(kQueueViewLimit) → TaskInfo{id,name,priority,
          effective_level = queue_.effective_level(*task, now), waited_ms = task-
>waited_ms(now)}
```

平均值的两个陷阱：

- 分母是 `started_`（**真正开始执行过**的任务数），不是 `submitted_`；否则队列里还有一堆没开始的任务时，

平均值会被稀释得毫无意义。

- `avg_run_ms` 的分母用同一个 `started_`，因为"执行过的任务"与"完成的任务"在同一时刻几乎相等，

而 `completed_` 会随失败/取消漏掉一部分。

`uptime_seconds` 的小问题： `started_at_` 在一次 `shutdown()` 之后不会清零，

所以关闭之后界面上的"运行 N 秒"仍会继续增长。这是已知的展示瑕疵，不影响统计。

5.9.11 `reset_counters()`（第 456–475 行）

```
{
    std::lock_guard<std::mutex> lock(mutex_);
    submitted_/started_/completed_/failed_/cancelled_/rejected_/wait_ns_total_/run_ns_total_ =
0;
    for (auto& w : workers_) w->reset_stats();
}
events_.add(EventKind::Info, "计数器已清零");
```

为什么清零也要持锁： 工作线程是在锁内 `mark_idle()` / `note_executed()` 的，

如果这里无锁清零，可能把它正在累加的那一笔抹掉——分子清了一半、分母没清，平均值就会漂。

5.9.12 `worker_main()`（第 490–521 行）

```
for (;;) {
    锁内：
        cv_.wait(lock, [&]{ return worker.retire_requested || stopping_ || !queue_.empty(); });
        if (worker.retire_requested) break;                    // 缩容：退休
        if (stopping_ && (!drain_ || queue_.empty())) break;   // 关闭：drain 时要把队列吃干净
        task = queue_.pop();
        if (!task) continue;                                   // 被别的线程抢先拿走了
        worker.mark_busy(task->id);
        task->state = Running;
        task->started_ns = to_ns(now);
        running_tasks_[task->id] = task;                       // 登记！cancel() 靠它找回任务
    锁外：
        run_task(worker, task);
    锁内：
        running_tasks_.erase(task->id);
}
```

"先登记再执行"是取消机制的关键： 任务一旦离开队列，`cancel()` 就只剩 `running_tasks_` 能找回它。

如果登记晚于执行，就会有一段"取消不到"的窗口。

5.9.13 `run_task()`（第 524–586 行）

```
① 取消检查①（开跑前）：cancel_requested 为真
       → state = Cancelled、finished_ns = now、cancelled_++、mark_idle()、
         记 Cancel「开始执行前发现已取消」，return
② started_++；等待时长 = started_ns - submitted_at → wait_ns_total_ +=
③ sampled = should_log_task(id)；sampled 时记 Start 事件
```

```
④ t0 = now；try { task->work(); } catch(...) { ok = false; }；t1 = now
⑤ 取消检查②（收工后）：cancelled = task->cancel_observed
⑥ run_ms = t1 - t0；finished_ns = t1；state = cancelled ? Cancelled : (ok ? Succeeded : Failed)
⑦ mark_idle()；note_executed(run_ms)；run_ns_total_ += run_ms * 1e6
⑧ cancelled → cancelled_++ + Cancel 事件
   ok        → completed_++ + Finish 事件（采样）
   否则      → failed_++ + Fail 事件（不采样）
```

为什么取消检查②看的是 `cancel_observed` 而不是 `cancel_requested`：

只有任务体**真的轮询到了**取消，才算"取消成功"；压根没理取消请求、把活干完的任务，

仍然记 `Succeeded`——谎报 `Cancelled` 比不取消更难查。

异常隔离： `catch (...)` 把一切异常吞在工作线程里。任务体抛异常只是这个任务 `Failed`，

线程本身继续用；否则整个池子会少一个线程。

5.9.14 `monitor_main()`（第 591–609 行）

```
for (;;) {
    锁内：
        monitor_cv_.wait_for(lock, kMonitorTick, [this]{ return monitor_stop_; });
        if (monitor_stop_) break;
        if (!running_ || stopping_) break;
        maybe_scale_locked();
    锁外：
        gone = take_retiring();
        逐个 join（【易错点】必须锁外）
}
```

`wait_for` 带谓词：既会被 `notify` 提前唤醒，也会每 100ms 自己醒一次。

5.9.15 `aging_rebuild_interval()`（第 613–620 行）

```
const double rate = config_.aging_levels_per_second;      // 级/秒
if (rate <= 0.0) return std::chrono::hours(1);            // 老化关了 → 基本不重建
const double seconds = kAgingRebuildLevelStep / rate;      // 攒够 0.1 级要多久
auto interval = milliseconds(seconds * 1000 + 0.5);
return interval < kMonitorTick ? kMonitorTick : interval;  // 不低于一个心跳
```

默认 0.2 级/秒 ⇒ 0.5 秒一次。老化速度调快，间隔自动缩短；调慢则更省。

5.9.16 `maybe_scale_locked()`（第 623–680 行）— 扩容 / 缩容 / 老化

```
now/pending/total/busy/idle 全部取当前值
老化：aging 开启且距上次重建 ≥ aging_rebuild_interval() → queue_.rebuild()、记时间

扩容（三个条件同时成立）：
    pending >= scale_up_threshold
    且 total < max_threads
    且 since_last_scale >= scale_cooldown_seconds
  → add = min(scale_up_step, max_threads - total)，再 max(add,1)
  → spawn_workers_locked(add)；last_scale_at_ = now；idle_since_ = {}
  → 记 ScaleUp「扩容 +N → M 线程（积压 K）」；return（这一跳不再考虑缩容）

缩容：
  can_shrink = total > min_threads && idle >= idle_workers_to_shrink && pending == 0
  !can_shrink → idle_since_ = {}; return          // 不满足就重置计时（滞回）
```

```
idle_since_ 为空 → idle_since_ = now
idle_for = now - idle_since_；< idle_timeout_seconds → return
since_last_scale < scale_cooldown_seconds → return
shrink = min(scale_down_step, total - min_threads)，再 max(shrink,1)
→ retire_workers_locked(shrink)；last_scale_at_ = now；idle_since_ = {}
→ 记 ScaleDown「缩容 -N → M 线程（缩容前空闲 K）」
```

为什么这么设计：

- **扩容按"积压"而不是"忙碌比例"**：积压是"用户已经在等"的直接证据，比忙碌比例更不容易抖。
- **缩容有三个额外条件**：不能低于下限、空闲线程要够多、队列必须空——少一个都会导致"刚缩完又来任务，立刻再扩"的抖动。再加 `idle_timeout_seconds` 的滞回，空闲计时中途只要不满足就清零重来。

5.9.17 `busy_count_locked()`（第 682–688 行）

遍历 `workers_` 数 `w->busy()`。O(线程数)，每 100ms 一次，可以忽略。

为什么不维护一个 `busy_` 计数：那样每次 `mark_busy/mark_idle` 都要动共享状态，还得处理"线程退休时 busy 归谁"的问题；直接数一遍更简单，也不慢。

5.9.18 `spawn_workers_locked()`（第 691–700 行）

```
auto worker = std::unique_ptr<Worker>(new Worker(next_worker_index_++));
worker->launch([this](Worker& w) { worker_main(w); },
               [this]() { cv_.notify_all(); });   // 析构兜底的唤醒器
workers_.push_back(std::move(worker));
```

第二个参数就是 5.5 里说的 `wake_on_retire_`：万一有线程没被 notify 就要析构，它能保证 `join()` 不会永远等下去。

5.9.19 `retire_workers_locked()`（第 703–729 行）

```
第一轮：从前往后挑 !busy() 的线程 → 置 retire_requested、move 进 retiring_、从 workers_ 删除
        （空闲线程能立刻退出，不会打断正在执行的任务）
第二轮：空闲的还不够 → 从末尾无条件挑（可能是忙的），等它干完当前任务自然退出
最后：retired > 0 → cv_.notify_all()，叫醒这些线程让它们看见退休标志
```

第二轮为什么允许挑忙线程：如果所有线程都在跑长任务，"等空闲再缩容"就永远等不到。标记退休后线程会在**干完当前任务之后**退出，不打断任务，只是不再取新任务。

5.9.20 `take_retiring()`（第 731–736 行）

锁内 `out.swap(retiring_)`，返回给调用方在**锁外** join。这是"锁内搬、锁外等"这一模式的收口。

5.10 `app/dashboard.hpp`（89 行）+ `app/dashboard.cpp`（673 行）— 仪表盘

职责：把 `PoolSnapshot` 画成五块面板，并把用户的鼠标操作翻译成对线程池的调用。

它不持有任何线程，也不做任何调度——所有"改状态"的动作都通过 `pool.` 的公开接口完成。

5.10.1 成员变量：三类

(1) 配置与开关

成员	类型	作用
`cfg_`	`tp::PoolConfig`	面板上**正在编辑**的配置（可能是非法值，靠滑块联动约束）
`resync_config_`	`bool`	"下一帧用线程池里的生效值回写面板"的标记
`live_apply_`	`bool`	「改动立即生效」复选框（默认关：拖滑块不会狂发 `update_config`）
`full_trace_`	`bool`	「记录所有任务事件」复选框

(2) 负载发生器

成员	作用
`factory_`	`tp::WorkloadFactory`，构造时给固定种子
`workload_kind_`	0=CPU / 1=IO / 2=Mixed（对应 `Combo` 下标）
`priority_mode_`	0=随机，1..4 = 低/普通/高/紧急（映射到 `priority_from_slot(mode-1)`）
`intensity_`	任务强度倍数（0.2–4.0）
`batch_size_`	「提交一批」一次提交几个（1–200）
`sustain_` / `rate_per_second_` / `submit_accumulator_`	持续压测开关、目标速率、按帧累积的小数余量
`submitted_by_ui_`	界面自己发过的任务数，用来给任务命名（`Mixed-1957`）

(3) 曲线与事件缓存

成员	作用
`hist_threads_` / `hist_queue_` / `hist_wait_` / `hist_throughput_`	四条曲线，各 `std::vector<float>`，最多 240 点（0.15s 一个点 ≈ 36 秒窗口），连续存放以便直接画
`last_sample_at_` / `last_completed_` / `throughput_`	采样时刻、上次完成数、算出来的吞吐
`events_cache_`	最近一次拉取的事件（`std::vector<PoolEvent>`），避免每帧都去拿 EventLog 的锁
`last_events_at_` / `last_event_at_` / `scroll_events_`	拉取节流（0.2s）、判断"有没有新事件"、是否自动滚到底
`events_total_` / `events_dropped_`	事件总数与覆盖数（界面上显示"累计 N 条 · 已覆盖 M 条"）

5.10.2 匿名命名空间里的 6 个小工具（第 18–78 行）

函数	作用

函数	作用
`float sc(float value)`	所有像素尺寸乘 DPI 缩放（`ui_scale()`），界面才会跟着系统缩放等比放大
`ImVec4 priority_color(tp::Priority)`	优先级 → 颜色（低=灰、普通=白、高=橙、紧急=红）
`ImVec4 event_color(tp::EventKind)`	事件种类 → 颜色（完成=绿、失败/拒绝=红、取消/缩容=橙、扩容=青）
`void metric_card(label, value, color)`	指标卡：上面小字标签、下面大字数值（数值用 title 字体）
`void section(const char* title)`	分组标题：一行间距 + 彩色标题 + 分隔线
`void plot_series(label, vector, vmin, vmax)`	一条曲线：空数据时显示"等待数据…"，否则 `ImGui::PlotLines`

> 四条曲线都用 `std::vector<float>`（容量 240，`reserve` 过），`plot_series` 直接把 `data.data()` 交给 `PlotLines`，**每帧零拷贝、零分配**。
>
> 早期版本用的是 `std::deque`，而 `PlotLines` 只吃连续内存，于是每帧都要临时拷一个 `vector`（4 条 × 240 点 ≈ 4 KB，60 帧/秒 ≈ 230 KB/s 的堆分配抖动）——现已改掉。
>
> 窗口满 240 点后每次采样整体左移一格（`std::copy`），采样频率 6.7 次/秒，代价可忽略。

5.10.3 绘制函数

函数	行号	做什么
`Dashboard::Dashboard()`	80–85	只做一件事：`factory_(20251029u)`（固定种子，两次运行可复现）
`Dashboard::enable_demo()`	87–97	`--demo` 用：混合负载、随机优先级、每批 20 个、450 任务/秒、自动持续压测
`Dashboard::draw(tp::ThreadPool&)`	99–148	**每帧的入口**：铺满视口建根窗口 → 算 `dt`（异常跳变时回落 1/60）→ 驱动负载发生器 → **抓一次快照** → 需要则回写 `cfg_` → 采样曲线 + 刷新事件 → 顶栏 → 左栏（配置）→ 右栏（指标/网格/曲线/表格）
`draw_top_bar(pool, snap)`	150–206	标题、运行状态、运行秒数与 FPS；右侧按钮：启动 / 优雅关闭 / 立即停止 / 清零统计 / 清空日志 / 重置配置
`draw_config_panel(pool, snap)`	208–363	全部滑块与复选框 + 负载发生器；滑块上下界互相联动且 `AlwaysClamp`；底部显示"生效：线程 A~B · 阈值 T · 容量 C"和校验错误
`draw_metrics(snap)`	365–413	六张指标卡：活跃线程（忙/总）、队列积压、已完成、失败/拒绝、平均等待、吞吐量
`draw_thread_grid(snap)`	415–483	每个线程一个方块（橙=忙、灰=

函数	行号	做什么
		闲、红=退休中），按可用宽度自动换行；悬停显示当前任务、累计执行数、累计忙碌时长
`draw_charts(snap)`	485–511	四条曲线：活跃线程数（纵轴上限跟着 `max_threads` 走）、队列积压、平均等待、吞吐
`draw_tables(snap)`	513–601	底部两张表：左「等待队列」前 64 条（ID / 任务名 / 优先级 / 有效优先级 / 已等待），右「事件日志」最近 200 条（自动滚到底）

`draw()` 里最要紧的一行是 `const tp::PoolSnapshot snap = pool.snapshot();`：

整帧只抓一次，后面所有面板都读这一份快照。这样一帧之内的数字是**自洽**的

（不会出现"线程数这一栏是 4、那一栏是 9"），也把加锁次数压到每帧一次。

5.10.4 数据流函数

函数	行号	做什么
`handle_load_generator(pool, dt)`	603–612	持续压测：`accumulator += dt × rate`，取整数部分提交，单帧最多 200 个（界面卡了一下不会一次灌几千个）
`submit_one(pool)`	614–624	掷一次优先级（随机或按下拉框），拼任务名 `"Mixed-1957"`，`factory_.make(kind, intensity_)` 造任务体，`pool.submit(...)`
`sample_history(snap, now)`	626–657	每 0.15s 采一个点：`throughput = Δcompleted / Δt`，四条曲线各 push 一个值，超过 240 点弹掉最旧的
`refresh_events(pool, now)`	659–662	每 0.2s 拉一次 `events().tail(kEventViewLimit)` + `total()` + `dropped()`；最新一条时间变了就标记"滚到底"

为什么要节流：`snapshot()` 要拿线程池的锁、`tail()` 要拿 EventLog 的锁。

以 60 帧/秒直接调，等于每秒抢 120 次锁；而人类眼睛只需要 0.2 秒更新一次事件列表。

曲线采样 0.15s 也是同理——但**指标卡必须每帧更新**（用户盯着数字变化时，卡顿很明显）。

5.10.5 界面上几个"看起来是 UI，其实是并发"的地方

现象	背后的机制
拖「最多线程」滑块，运行时立刻多出一个方块	`changed && live_apply_ && running` →

现象	背后的机制
	`update_config()` → `spawn_workers_locked`
点「优雅关闭」，线程数慢慢降到 0	`shutdown(true)`：先跑完队列，再 notify + join（join 在锁外）
点「立即停止」，队列一下子清空、出现一片 Cancel	`shutdown(false)`：非优雅路径把排队任务全部标 `Cancelled`
线程方块变红	`retire_requested` 已置位但线程还在跑完当前任务 → 界面显示"退休中"
「生效：线程 4~9」和滑块上的数字不同	滑块编辑的是 `cfg_`；这一行读的是 `snap.config`（夹紧后的真实值）

5.11 `app/theme.hpp`（53 行）+ `app/theme.cpp`（187 行）— 配色与中文字体

职责：把"好看"从"画什么"里剥离出来。`dashboard.cpp` 只说"用 `pal.accent` 画标题"，

具体是什么颜色不在那里决定。

5.11.1 数据结构

```
struct Palette {          // 13 个颜色
    ImVec4 window_bg, panel_bg, panel_bg2, border;
    ImVec4 text, text_dim, accent, ok, warn, err;
    ImVec4 busy, idle, retiring;   // 线程方块的三种状态色
};
struct Fonts { ImFont* body = nullptr; ImFont* title = nullptr; };
```

接口	说明
`const Palette& palette()`	全局配色表
`const Fonts& fonts()`	两级字体（正文 17px、标题 25px，都乘 DPI 缩放）
`float ui_scale()`	当前 DPI 缩放（`dashboard.cpp` 的 `sc()` 依赖它）
`ImU32 u32(const ImVec4&)`	颜色转 `ImU32`，给 `ImDrawList::AddRectFilled/AddText` 这类底层 API 用
`void apply_theme(float scale)`	设 `ImGuiStyle`（圆角、间距、边框）+ 逐条填 `style.Colors` + `ScaleAllSizes(scale)`
`bool load_fonts(float scale)`	找系统中文字体、烘焙字形图集

5.11.2 `apply_theme()`：只配一次，且 `ScaleAllSizes` 只能调一次

```
if (scale < 0.75f) scale = 0.75f;
if (scale > 3.0f) scale = 3.0f;
ImGui::StyleColorsDark();          // 先继承 ImGui 的默认深色
... 逐项覆盖 rounding / padding / spacing / 40 多个颜色 ...
style.ScaleAllSizes(scale);        // 最后一次性按 DPI 放大所有像素尺寸
```

为什么 `ScaleAllSizes` 只能执行一次：它会把 `style` 里所有像素尺寸乘以 `scale`。

重复调用就是反复乘，界面会越开越大。所以 `apply_theme()` 全程只在启动时调一次。

5.11.3 `load_fonts()`：字体是整块贴图，字少了才快

```
① 按顺序探测 4 个候选字体：msyh.ttc → msyhl.ttc → simhei.ttf → simsun.ttc
   全都没有 → AddFontDefault()（英文字体，中文会显示成 ?），返回 false
② 字形范围 = GetGlyphRangesChineseSimplifiedCommon()（约 2500 常用字）
   + kExtraGlyphs 里手补的界面专用字（"▲►▼→兜函屏崩帧戳拟拷晰栅槛渲滞焙粹躯辑迭逻钮阈频馈"）
③ 用 ImFontGlyphRangesBuilder 合并成一份静态 ImVector<ImWchar>
④ 正文 17px、标题 25px 各加一次（都乘 scale），开 OversampleH=2 / PixelSnapH /
RasterizerMultiply=1.06
⑤ io.FontDefault = body
```

三个坑，都写在注释里了：

1. **`static ImVector<ImWchar> ranges` 必须是静态的**：ImGui 的字体贴图在 `load_fonts()` 返回之后才真正烘焙，`ranges.Data` 那一刻必须还活着。

2. **不用 `GetGlyphRangesChineseFull()`**（约 2 万个汉字）：那要烘一张 8K 级贴图，显存和启动时间翻几倍，而界面上的字是固定的。

3. **新加的中文显示成 `?`**：把那个字补进 `kExtraGlyphs` 就行（这是最省事的兜底办法）。

5.12 `app/main.cpp`（287 行）— 平台层：窗口 + D3D11 + 主循环

职责：整个项目里**唯一**跟 Windows API 打交道的地方。它做四件事，做完就退场：

1. 用 Win32 造一个窗口；
2. 用 D3D11 造一个能画的表面（交换链 + 渲染目标）；
3. 把 Dear ImGui 挂到这个表面上（`imgui_impl_win32` + `imgui_impl_dx11`）；
4. 跑一个循环：`NewFrame → dashboard.draw(pool) → Render → Present`。

一句话类比：ImGui 自己不会开窗口，它只在"别人的"绘制上下文上画三角形。

所以先用 Win32 造窗口，再用 D3D11 造渲染目标，最后把 ImGui 挂上去。

5.12.1 全局对象（第 36–43 行）

全局	类型	说明
`g_device`	`ID3D11Device*`	D3D11 设备（创建资源）
`g_context`	`ID3D11DeviceContext*`	设备上下文（下命令）
`g_swap_chain`	`IDXGISwapChain*`	交换链（双缓冲，`Present` 时提交给屏幕）
`g_render_target`	`ID3D11RenderTargetView*`	渲染目标视图（**可能为 null**，所有使用点都要判）
`g_running`	`bool`	主循环开关，收到 `WM_DESTROY` 置 false
`g_pending_width/height`	`UINT`	窗口刚被拖动过、待重建的尺寸（0 表示不需要重建）

为什么用全局对象而不是一个类：D3D11 的生命周期与进程一致，`wnd_proc` 是 C 回调拿不到 `this`。

教学项目里这样最直白；真要做大就包一个 `Renderer` 类，把 `wnd_proc` 的 `GWLP_USERDATA` 指过去。

5.12.2 `create_render_target()`（第 48–59 行）

```
ID3D11Texture2D* back_buffer = nullptr;
const HRESULT hr = g_swap_chain->GetBuffer(0, IID_PPV_ARGS(&back_buffer));
if (FAILED(hr) || !back_buffer) return false;              // ① 取后备缓冲
const HRESULT view_hr = g_device->CreateRenderTargetView(back_buffer, nullptr,
&g_render_target);
back_buffer->Release();                                    // ② 包一层 RTV
return SUCCEEDED(view_hr) && g_render_target != nullptr;   // ③ 失败必须让调用方知道
```

两个 HRESULT 都要查。早期版本只查了第一个，`CreateRenderTargetView` 失败时 `g_render_target` 还是 null，主循环照样 `ClearRenderTargetView(nullptr, ...)`——对空 COM 指针调成员函数，直接访问违例（在 Debug 下是"看起来什么都没发生、然后窗口闪一下没了"）。

5.12.3 `create_device(HWND)`（第 71–105 行）

```
① 填 DXGI_SWAP_CHAIN_DESC：2 个缓冲、格式 R8G8B8A8_UNORM、窗口化、
   SwapEffect = DISCARD、SampleDesc.Count = 1（ImGui 自己抗锯齿，不要 MSAA）
② 先试 D3D_DRIVER_TYPE_HARDWARE；失败回落 D3D_DRIVER_TYPE_WARP（软件光栅化）
③ 两者都失败 → 打印 HRESULT 并返回 false
④ 成功 → return create_render_target();
```

WARP 回落的实际价值：远程桌面、虚拟机、老显卡上硬件设备创建常常失败；退回软件渲染虽然慢，但至少能起来（本项目就是这么在没显卡的机器上跑通的）。

5.12.4 `destroy_device()`（第 107–121 行）

按创建的反序 `Release()`：渲染目标 → 交换链 → 上下文 → 设备，每个指针都判空并置 null。

COM 是引用计数：谁 `AddRef` 谁 `Release`，这里只是把初始化时拿到的四个引用还回去。

5.12.5 `wnd_proc()`（第 127–152 行）

```
if (ImGui_ImplWin32_WndProcHandler(hwnd, msg, wparam, lparam)) return true;   // ① ImGui 优先
switch (msg) {
    case WM_SIZE:       // ② 只记尺寸，不在消息里重建缓冲
        if (wparam != SIZE_MINIMIZED) { g_pending_width = LOWORD(lparam); g_pending_height =
HIWORD(lparam); }
        return 0;
    case WM_SYSCOMMAND: // ③ 屏蔽 Alt 呼出菜单（否则按 Alt 会卡住输入）
        if ((wparam & 0xFFF0) == SC_KEYMENU) return 0;
        break;
    case WM_CLOSE:  ::DestroyWindow(hwnd); return 0;    // ④ 触发 WM_DESTROY
    case WM_DESTROY: ::PostQuitMessage(0); g_running = false; return 0;
}
return ::DefWindowProcW(hwnd, msg, wparam, lparam);
```

为什么把尺寸变动记成 `g_pending_*` 而不是当场 `ResizeBuffers`：

`ResizeBuffers` 要求"所有对后备缓冲的引用都已释放"，而在 `WM_SIZE` 里，

这一帧的 GPU 命令可能还没提交完。放到主循环开头（`Present` 之后、`NewFrame` 之前）做最安全。

5.12.6 `system_ui_scale(HWND)`（第 155–159 行）

```
float scale = ImGui_ImplWin32_GetDpiScaleForHwnd(hwnd);   // 100% → 1.0, 150% → 1.5
if (scale < 0.75f || scale > 3.0f) scale = 1.0f;           // 离谱的值直接回落到 1.0
return scale;
```

必须在窗口建好之后调用——窗口还没创建时问不到它所在显示器。

5.12.7 `wWinMain()`（第 166–287 行）— 六个阶段

阶段	行号	做什么
① 命令行	168	`std::wcsstr(cmd_line, L"--demo")` 判断是否演示模式
② 建窗口	169–198	`EnableDpiAwareness()` → 注册窗口类（`CS_OWNDC`）→ 按屏幕尺寸算窗口大小（至少 1280×820）→ `CreateWindowExW`
③ 建设备	200–208	`create_device()` 失败就弹 `MessageBox` 并退出
④ 初始化 ImGui	210–220	`CreateContext` → 取 DPI → `io.IniFilename = nullptr`（不往仓库写 `imgui.ini`）→ `apply_theme` → `load_fonts` → 两个后端 `Init`
⑤ 主循环	231–276	见下
⑥ 收尾	278–286	`pool.shutdown(true)` → 两个后端 `Shutdown` → `DestroyContext` → `destroy_device` → 销毁窗口类

主循环每帧的顺序（顺序错了就会闪）：

```
PeekMessage 循环（Translate + Dispatch；收到 WM_QUIT 就退出）
IsIconic 最小化 → Sleep(10) 继续（不空转烧 CPU）
有 pending 尺寸 → cleanup_render_target() → ResizeBuffers() → create_render_target()
                    任何一步失败 → 打印并 break（走正常收尾，不拿 nullptr 继续画）
ImGui_ImplDX11_NewFrame(); ImGui_ImplWin32_NewFrame(); ImGui::NewFrame();
dashboard.draw(pool);              // ← 整个界面就在这一行
ImGui::Render();
if (!g_render_target) break;       // 兜底
OMSetRenderTargets + ClearRenderTargetView(g_render_target, 背景色)
ImGui_ImplDX11_RenderDrawData(GetDrawData())
g_swap_chain->Present(1, 0);       // 1 = 开垂直同步：不撕裂、也不白烧 GPU
```

`Present(1, 0)` 的 `1` 是同步间隔：等一个垂直同步周期再翻页，于是帧率被钉在刷新率上（实测 60Hz 屏上 60 FPS 左右）。想测极限帧率可以传 0，但考核项目里没必要。

5.12.8 想搬到 Linux 要改哪些

`app/main.cpp` 是**唯一需要重写**的文件（约 150–200 行）：Win32 窗口 → SDL2/GLFW，D3D11 → OpenGL3，`imgui_impl_win32/dx11` → `imgui_impl_sdl2/opengl3`。

`dashboard.cpp`、`theme.cpp`、整个 `tp_core` 一行都不用改——它们不知道自己在哪个系统上跑。

5.13 `apps/cli_main.cpp`（285 行）— 无界面压测程序

职责：在没有窗口、没有显卡的环境里验证线程池的真实行为。它是**最便宜的验收工具**：CI 里跑它、改完内核先跑它，比开界面快得多。

流程：分批提交 → 边跑边打印状态 → 等缩容 → 打最终报告 → 打最近事件。

> **为什么输出全是英文/ASCII**：Windows 控制台默认代码页 936（GBK），源码里的中文直接 `printf` 出来会变乱码。文件头注释里专门记了这条踩坑。

5.13.1 `struct Options`（第 38–45 行）

字段	默认	说明
`tasks`	300	总任务数
`min_threads`	2	初始/最少线程
`max_threads`	8	上限
`batch`	25	每批提交数
`batch_pause_ms`	20	批间停顿（给监控线程留出观察积压的时间）
`workload`	Mixed	负载种类

5.13.2 `parse_workload()`（第 59–64 行）

`"cpu"/"io"/"mixed"` → 枚举，大小写敏感；不认识的字符串返回 false（由调用方打印用法并退出 1）。

5.13.3 `parse_size()`（第 66–81 行）— 一个"看起来很无聊但全是坑"的函数

```
if (!text || text[0] == '\0') return fallback;
if (!std::isdigit(static_cast<unsigned char>(text[0]))) return fallback;  // 挡住 "-5" 与前导空白
errno = 0;
const unsigned long long v = std::strtoull(text, &end, 10);
if (end == text || *end != '\0' || errno == ERANGE) return fallback;      // 挡住 "12abc" / 溢出
if (v > std::numeric_limits<std::size_t>::max()) return fallback;
return static_cast<std::size_t>(v);
```

为什么不能直接用 `strtoul`：

1. `strtoul` 接受负号，`--tasks -5` 会**回绕**成 `18446744073709551611`，

程序会试图提交 1.8e19 个任务；

2. 超出 `ULONG_MAX` 时它只返回上限并置 `errno = ERANGE`，不查就是一个天文数字；

3. 它还会跳过前导空白、接受 `+5`。

现在的策略是：**任何不合法的输入都回落默认值，不报错退出**。

理由是这个程序是给人手敲的，"我敲错了"不应该让整场压测白跑。

5.13.4 `parse_args()`（第 84–125 行）

手写循环，**不引入 getopt**（教学项目里少一个依赖就少一个坑）：

参数	处理
`--help` / `-h`	置 `want_help` 并立即返回 true
`--tasks/--threads/--max/--batch`	取下一个 argv 作为值；`--batch` 额外 `max(1, ...)`
`--pause`	取不到值就返回 false；值转 `int`
`--workload`	`parse_workload()` 失败 → 返回 false
其它	打印 `unknown argument` 到 stderr，返回 false

最后两条兜底：`min_threads` 至少 1；`max_threads < min_threads` 时抬到 `min_threads`。

注意 `--pause` 没有下限保护（`--pause 0` 会变成忙等提交），

这是这个参数可以接受的粗糙之处：它只影响压测节奏，不影响正确性。

5.13.5 `print_progress()`（第 128–136 行）

每 250ms 打一行：

```
t=  1.25s  threads= 6 (peak  6) busy=  6 queue= 12 | done= 188 fail=0 rej=0 | avg_wait=
3.21ms avg_run=  12.40ms
```

`max_seen` 是调用方维护的峰值（`std::max(peak, snap.threads)`），方便一眼看出"到底扩到过几个线程"。

5.13.6 `main()`（第 140–285 行）— 七个步骤

步骤	行号	做什么
① 解析参数	141–150	失败打用法返回 1；`--help` 返回 0
② 准备配置	152–174	`queue_capacity=4096`、`threshold=8`、`step=2`、`cooldown=0.2`、`down_step=2`、`idle_need=2`、`idle_timeout=0.8`、开老化；`start()` 失败返回 2
③ 分批提交	176–194	每批 `min(batch, 剩余)` 个，任务名 `"Mixed-<序号>"`，提交失败打印 `[backpressure] queue full,`

步骤	行号	做什么
		`task rejected`（这是**背压**，不是错误）；每批后 `sleep(batch_pause_ms)`
④ 轮询等待	196–213	每 20ms 抓一次快照；每 250ms 打一行进度；完成判据见下；**60 秒硬超时**
⑤ 等缩容	215–222	最多 10 秒，等 `threads <= min_threads && queue_size == 0`（观察空闲回收）
⑥ 最终报告	224–256	耗时、线程数/峰值/范围、submitted/completed/failed/cancelled/rejected、平均等待/执行、吞吐；最后打印最近 12 条事件
⑦ 结论与退出码	258–284	`rejected > 0` 或"没全部结清"→ 打印告警并 **exit 1**；全部跑完 → 打印 `result: all N task(s) ran` 并 exit 0

完成判据（曾经写过一版错的，已修）：

```
const std::uint64_t done = snap.completed + snap.failed + snap.cancelled;
if (done + snap.rejected >= opt.tasks) break;   // 所有"尝试过的提交"都结清了
```

`done` 只统计**真的进过队列、执行过**的任务；`rejected` 是被背压挡在门外、从未执行的任务，

所以它只用来判断"提交额度是否已经用完"，**不算完成**。

报告阶段的结论逻辑是：

```
int exit_code = 0;                          // 0 = 这一轮压测跑全了
if (final_snap.rejected > 0) {
    // 被背压挡在门外的任务：从没跑过，不能算"跑完了"
    std::printf("[warning] %llu task(s) rejected, never ran\n", ...);
    exit_code = 1;
}
if (accounted < opt.tasks) {
    std::printf("[warning] %llu of %zu accounted (timeout)\n", ...);
    exit_code = 1;
}
// failed > 0 只提示：异常被池隔离，池本身是健康的
if (exit_code == 0) std::printf("result: all %zu task(s) ran\n", opt.tasks);
return exit_code;
```

> 早期版本是 `finished = completed + failed + cancelled + rejected`，把被拒任务也算成"完成"，
>
> 于是队列 4096 打满时 CLI 会提前宣布"跑完了"，而实际上有一批任务一次都没执行过。
>
> 现在被拒任务单独告警，并且 **CLI 以非零退出码**告诉你这一轮压测没跑全 ——
>
> 拿它做脚本或 CI 时，失败不再被吞掉。

其他值得一提的点：

- `submitted += n` 是无条件推进的（拒绝后不重试）——保压测节奏，不追求"每个任务都执行"。
- 吞吐用 `completed / 总耗时` 算，包含提交与等待缩容的时间，所以数字偏保守。
- 结束前调 `pool.shutdown(true)`，把队列里可能剩下的任务跑完再退出。

5.14 `tests/` — 手写测试框架 + 22 个用例

为什么不用 gtest/catch2：教学项目尽量零依赖，而且"自己写一遍测试框架"本身就是很好的练习——

注册、断言、异常隔离、统计，四件事一共 113 行。

5.14.1 `tests/test_framework.hpp`（113 行）

组件	签名 / 形式	作用
`TestCase`	`struct TestCase { const char* name; void (*fn)(); };`	一个用例 = 名字 + 函数指针
`registry()`	`std::vector<TestCase>& registry()`	全局用例表（函数内 `static`，避开初始化顺序问题）
`check_count()` / `fail_count()`	`std::size_t& ...()`	全局断言总数 / 失败数
`current_name()`	`const char*& current_name()`	当前正在跑的用例名（失败信息里带上它）
`Registrar`	`Registrar(const char* name, void (*fn)())`	构造函数里往 `registry()` push 一条——**静态对象在 main 之前完成注册**
`show(ok, expr, file, line, expected, actual)`	记录一次断言	成功只计数；失败打印 `[FAIL] ... at file:line`
`report_failure(...)`	输出到 **stderr**	失败详情（期望值 / 实际值）
`run_all()`	`int run_all()`	遍历注册表；每个用例单独 `try/catch`（`std::exception` 与 `...` 都接住），**用 `fail_count()` 的差值**判断该用例是否通过；返回失败数（>0 → 进程退出码非 0）
`TF_TEST(name)`	宏	展开成：函数前置声明 + 静态 `Registrar` + 函数定义
`TF_CHECK(cond)`	宏	条件为假记一次失败，**不中断**该用例（一次跑出所有问题）
`TF_CHECK_EQ(a, b)`	宏	`const auto& tf_a = (a); const auto& tf_b = (b);` 再比较——避免宏参数被求值两次（`++i` 之类）

`TF_TEST` 的展开值得记住（这是"静态注册"的经典写法）：

```
#define TF_TEST(name)                                          \
    static void name();                                        \
    static ::tftest::Registrar tf_reg_##name(#name, &name);  \
```

```
    static void name()
```

于是 `TF_TEST(pool_runs_all_submitted_tasks) { ... }` 既定义了函数，又让它在 `main` 之前自动登记。

运行输出长这样：

```
[ OK ] queue_orders_by_priority          ( 5 checks)
[ OK ] pool_runs_all_submitted_tasks     ( 6 checks)
[FAIL] pool_cancel_after_finish_returns_false  ( 3 checks)
...
22 cases, 136 checks, 0 failures
```

5.14.2 `tests/test_main.cpp`（6 行）

```
#include "test_framework.hpp"
int main() { return tftest::run_all(); }
```

就这一件事：把所有注册的用例跑一遍，失败数作为退出码（CI/CTest 靠它判断红绿）。

5.14.3 `tests/test_priority_queue.cpp`（136 行）— 6 个队列用例

这一组**一个线程都没起**——队列刻意无锁，所以可以纯单线程测，跑得飞快。

#	用例（行号）	验证什么	关键手法
1	`queue_orders_by_priority`（33）	高优先级先出队	乱序 push，按 `Critical → High → Normal → Low` 断言 pop 顺序
2	`queue_is_fifo_for_same_priority`（57）	同优先级先进先出	同优先级连 push 3 个，断言按提交序号出队（比较器用 `sequence` 而不是指针）
3	`queue_aging_promotes_old_low_priority`（68）	老化能把低优先级提上来	关掉老化对照，再开 `levels_per_second=100.0` 放大时间效应
4	`queue_rejects_when_full`（89）	满容量返回 false（背压）	`capacity=2` 连 push 3 个，第三个失败且 `rejected_total()==1`
5	`queue_remove_by_id`（99）	按 id 摘除，其余顺序不变	push 3 个 → `remove(中间那个)` → 断言剩余顺序与 size
6	`queue_snapshot_is_sorted_and_limited`（118）	快照按紧急度降序且被截断	push 多个后 `snapshot(2)`，断言 size==2 且第一个是最高优先级

5.14.4 `tests/test_pool.cpp`（544 行）— 16 个线程池用例

先认识两个辅助件（文件顶部）：

- **`wait_until(pred, timeout)`**：每 5ms 轮询一次谓词，直到为真或超时。全部并发用例都靠它，

不用固定 `sleep`（除了两处历史遗留，已改成确定性等待）。理由：`sleep(50ms)` 在高负载 CI 上照样会翻车。

• **gate / blocker 模式**：提交一个卡在条件变量上的"堵门任务"占住唯一的工作线程，这样后续任务一定停在队列里，才能稳定地测"排队中"的行为。

#	用例（行号）	验证什么	关键手法
1	`pool_runs_all_submitted_tasks`（36）	基本功能：提交 N 个都跑完	`wait_until(completed == N)`，断言 `failed==0`
2	`pool_runs_higher_priority_first`（58）	调度确实按优先级	`min=max=1`，先让 blocker 占住唯一线程，再按低→高顺序提交，放行后断言执行顺序
3	`pool_rejects_when_queue_is_full`（110）	队列满 → `submit` 返回 `nullopt`	容量设小 + blocker 占住线程，断言 `.has_value()` 为假
4	`pool_cancel_pending_task`（136）	排队中的任务能被摘掉	任务在队列里时 `cancel(id)` 为真，放行后该任务**从未执行**（用一个 `atomic<bool> ran` 断言）
5	`pool_survives_task_exception`（174）	任务抛异常不打死线程	提交一个 `throw std::runtime_error` 的任务，再提交一个正常任务，断言后者完成、`failed==1`
6	`pool_scales_up_under_backlog`（196）	积压触发扩容	阈值调小 + 一批长任务，`wait_until(threads > min)`，并检查出现过 `ScaleUp` 事件
7	`pool_scales_down_when_idle`（224）	空闲太久会缩容	负载结束后 `wait_until(threads <= min, 超时给足)`，检查 `ScaleDown` 事件
8	`pool_non_drain_shutdown_cancels_queue`（255）	立即关闭会丢弃排队任务	blocker 占住线程 + 排队若干，`shutdown(false)` 后断言 `cancelled > 0` 且线程全部退出
9	`pool_update_config_grows_threads_immediately`（284）	热更新能把下限调大并立刻补线程	`update_config` 后立刻 `wait_until(threads >= 新下限)`
10	`pool_cancels_running_task_cooperatively`（305）	**运行中**任务能被协作取消	1 线程 + 任务体循环轮询 `token.is_cancelled()`；`cancel()` 返回 true，任务体收到信号提前收工；断言 `cancelled==1 && completed==0`，且结束之后再 cancel 返回 false
11	`pool_cancel_after_finish_returns_false`	已经跑完的任务取消返回	等 `completed==1` 再

#	用例（行号）	验证什么	关键手法
	（348）	false	`cancel(id)`，断言 false
12	`pool_task_ignoring_cancel_still_succeeds`（371）	不理取消、跑完的任务仍记 Succeeded	长任务**不**轮询 token，运行中 `cancel()` 返回 true，但最终 `completed==1 && cancelled==0`（不谎报）
13	`pool_submit_event_never_after_start`（399）	事件顺序不倒挂	200 个 1ms 任务 + 大容量，把 `tail(512)` 里每个任务 id 的 Submit/Start/Finish 下标记进 `std::map`，断言 `Submit < Start < Finish` 且样本数 > 0
14	`pool_rejects_illegal_config_on_start`（458）	**非法原始配置拒绝启动**	`min=8, max=2` → `!start()`、`!running()`；同时 `snapshot().config.max_threads == 8`（夹到 min）；改成 `max=16` 后能启动
15	`worker_destructor_wakes_parked_thread`（485）	`~Worker` 能唤醒睡着的线程	直接建 `tp::Worker`，body 在自有条件变量上等到 `retire_requested`，`wake_on_retire` 负责 notify；作用域结束若不能返回就是死锁（CTest 有 60 秒超时兜底）
16	`pool_task_event_full_trace_logs_every_task`（511）	全量记录开关有效	提交 id=1 的任务（`1 & 0x1F != 0`，默认采样必漏），开全量后断言 Submit/Start/Finish 各 1 条

三个"工程性"用例值得单独说：

• #14 存在的理由：`config_` 已经被夹紧，拿它 `validate()` 永远通过，于是"非法配置返回 false"这条验收标准会悄悄变成空话。用例把这个坑钉死。

• #15 存在的理由：线程睡在 `cv_.wait` 上时，只置 `retire_requested` 是**唤醒不了**它的（谓词不会自己重新求值），`join()` 会永远等下去。这个用例专门测那条兜底路径。

• #13 存在的理由：`Submit` 事件早期是在锁外补记的，worker 可能已经跑完并记了 `Finish`，日志里就会出现"先完成后提交"。把采样挪进临界区后，这个用例保证它不再退回去。

5.14.5 想加一个新用例

```
#include "test_framework.hpp"
#include "threadpool/pool.hpp"
```

```
namespace {
bool wait_until(...);   // 直接从 test_pool.cpp 抄一份，或提到公共头里

TF_TEST(pool_xxx) {
    tp::PoolConfig cfg;
    cfg.min_threads = cfg.max_threads = 1;   // 并发用例尽量把线程数钉死，结果才确定
    tp::ThreadPool pool(cfg);
    TF_CHECK(pool.start());
    // ... 提交任务、等条件、断言 ...
    pool.shutdown(true);
    TF_CHECK(!pool.running());
}
}
```

三条经验：**① 线程数尽量钉死**（否则时序不稳定）；**② 等条件不要等时间**（`wait_until` 而不是 `sleep`）；

③ 每个用例自己 `shutdown`（析构虽然兜底，但显式关闭能让泄漏/死锁暴露在用例内部）。

5.15 `CMakeLists.txt`（118 行）— 构建脚本

职责：把 25 个源文件编成 4 个目标，并且保证"我们自己写的代码严格、第三方代码不插手"。

5.15.1 逐段说明

行号	内容	为什么这么写
1	`cmake_minimum_required(VERSION 3.20)`	用到 `IMPORTED` 目标与 `target_*` 的现代写法
7–10	`project(threadforge VERSION 0.1.0 …)`	项目名 + 版本 + 只启 C++
12–14	`CMAKE_CXX_STANDARD 17` / `REQUIRED` / `EXTENSIONS OFF`	要 `std::optional`；`EXTENSIONS OFF` 防止写出 GNU 方言（严格警告才有效）
16–18	没指定构建类型 → 默认 `Debug`	傻瓜化：直接 `cmake -S . -B build` 也能调试
20–22	`RUNTIME_OUTPUT_DIRECTORY = <build>/bin`	三个 exe 全在一个目录，VS Code/CLI 路径固定
24	`find_package(Threads REQUIRED)`	线程库在 Linux 上是 `-pthread`，Windows 上不需要
27–29	`threadforge_warnings` INTERFACE 目标	`-Wall -Wextra -Wpedantic -Wshadow -Wconversion`，**只挂在自己的目标上**；`INTERFACE` 表示它自己不产生编译单元，只是"一串选项"
31–38	`-ffile-prefix-map=${CMAKE_SOURCE_DIR}=.`（仅 MinGW）	中文目录下 `__FILE__` 是非 ASCII，`assert()` 展开会报 `Illegal byte sequence`；顺带让调试路径更短
43–54	`tp_core` 静态库：6 个源文件	`PUBLIC` 头文件目录 = `include/`（使用方自动继承）；`PUBLIC Threads::Threads`、`PRIVATE threadforge_warnings`；`_CRT_SECURE_NO_WARNINGS` / `NOMINMAX` /

行号	内容	为什么这么写
		`WIN32_LEAN_AND_MEAN` 挡住 Windows 头文件的噪声
59–71	`imgui` 静态库：4 个核心 + 2 个后端	固定版本、不联网；链接 `d3d11 dxgi d3dcompiler dwmapi gdi32 imm32 ole32`
76–91	`threadforge_app`（`WIN32`）	`WIN32` 让它用 `wWinMain`、不弹控制台；`UNICODE/_UNICODE` + `-municode` 是 MinGW 下宽字符入口的必需项
96–97	`threadforge_cli`	只链 `tp_core`——**证明内核不依赖界面**
102–112	`tp_tests` + `add_test` + `TIMEOUT 60`	并发用例一旦死锁，进程会一直卡住；超时让它变成"明确的失败"而不是让 CI 干等
117–118	`run_app` / `run_cli` 自定义目标	`cmake --build build/debug --target run_cli` 直接跑

5.15.2 依赖关系（一眼看清谁不能依赖谁）

```
threadforge_app ──► tp_core
                └──► imgui ──► (Windows 系统库)

threadforge_cli ──► tp_core          ← 只此一条，没有 imgui
tp_tests        ──► tp_core          ← 同上，测试不需要窗口
```

`tp_core` 的 `target_link_libraries` 里只有 `Threads::Threads`。

想验证"内核真的没有平台依赖"，只看这一行就够了。

5.15.3 常用命令

```
cmake -S . -B build/debug -G Ninja -DCMAKE_BUILD_TYPE=Debug   # 配置
cmake --build build/debug                                     # 编译
cmake --build build/debug --target run_cli                    # 编译并跑 CLI
ctest --test-dir build/debug --output-on-failure              # 跑测试
cmake --build build/debug --clean-first                       # 全量重编（改头文件后保险）
```

6. 并发约束与踩坑清单

这一节是读 `src/pool.cpp` 的"注意事项"。**每一条都对应源码里的一段注释**，

改代码时先对照一遍。

6.1 十条硬约束

#	约束	违反之后的症状
1	**锁内只搬任务，锁外才 `work()`**	一个 100ms 的任务会让整个池子停 100ms（`submit`/`pop` 全卡）
2	**`join()` 一律在锁外**	工作线程要拿 `mutex_` 才能退出 → 死锁，进程挂住（CTest 60 秒超时能兜住）
3	**`PriorityTaskQueue` 自己不加锁**	它的接口全是非线程安全的，只能在持 `mutex_` 时调用
4	**`EventLog` 可以嵌套在池锁里调用**	反过来才有问题：EventLog 绝不能回调线程池（当前实现从不回调）
5	**先登记 `running_tasks_` 再执行**	否则"已出队但还没登记"的任务，`cancel()` 找不到它
6	**`is_finished()` 依赖枚举顺序**	加新状态时插错位置 → 把"没结束"当"已结束"
7	**计数器用 `relaxed` 原子，但 `reset_counters()` 要持锁**	无锁清零与工作线程累加竞争 → 平均值漂移（分子清了一半）
8	**`sanitized()` 的边界必须与 `validate()` 一致**	不一致时某个分支变成死代码，"非法配置返回 false"成了空话
9	**取消是请求，不是命令**	用 `TerminateThread`/`pthread_cancel` → 锁不释放、栈对象不析构，坏得最难查
10	**`~Worker()` 必须 notify**	线程睡在 `cv_.wait` 上，只置标志永远不会醒 → `join()` 永远等

6.2 五个真实踩过的坑

坑	症状	根因与修法
① `clear()` 不清 `total_`	点了"清空日志"，累计数还在涨	语义不一致：现在两者一起归零
② 界面 `cfg_` 与线程池 `config_` 是两套	拖完滑块显示 4~9，实际生效 4~12	加了 `resync_config_`：应用配置后下一帧用 `snap.config` 回写面板，并显示"生效：…"一行
③ 每帧全量排序	队列满时界面每隔 16ms 卡一次	`snapshot()` 改成装饰-选择（1340 µs → 104 µs），见 5.3.4
④ `Submit` 事件在锁外补记	日志里出现"先完成、后提交"	采样与 `submitted_++` 一起挪进临界区，并加了用例 #13
⑤ `~Worker()` 只置标志不 notify	缩容偶发卡死	`launch()` 新增 `wake_on_retire` 回调，析构时先唤醒再 join，并加了

坑	症状	根因与修法
		用例 #15

6.3 写新代码时的自检清单

- 这个函数需要 `mutex_` 吗？如果需要，注释里有没有标 🔓？
- 里面有没有 `join()`、`sleep`、回调用户代码？有的话必须在锁外。
- 新加的状态变量，是"读多写少"（原子量）还是"要跟别的变量一起保持一致"（进锁）？
- 新增的 `PoolConfig` 字段，`validate()` 和 `sanitized()` **两边都改了吗**？
- 新增的事件种类，`to_string` / `event_label_zh` / `event_color` **三处都补了吗**？
- 界面上新加的中文字，`kExtraGlyphs` 里补了吗（否则显示成 `?`）？

附录 A：配置项速查（想调什么改哪里）

你的目的	改这些	观察什么
看扩容（积压触发）	`scale_up_threshold` 调小（如 2）、`scale_up_step` 调大	线程数上升、界面出现 `ScaleUp`
看缩容（空闲回收）	`idle_timeout_seconds` 调小（如 0.5）、`idle_workers_to_shrink` 调小	负载结束后线程数回落、出现 `ScaleDown`
想看抖动（反例）	`scale_cooldown_seconds=0` + `idle_timeout_seconds=0`	线程数在 min/max 之间来回跳
测背压	`queue_capacity` 调小（如 8）	CLI 打印 `[backpressure]`、界面"拒绝"计数上涨
测优先级与老化	`aging_levels_per_second` 调大（如 3.0）	低优先级任务不再被饿死；队列表里"有效优先级"随时间上升
测取消	界面提交一批长任务后点"立即停止"	队列瞬间清空、事件里一片 Cancel
测异常隔离	在 `simulator.cpp` 里让某类任务偶发 `throw`	"失败"计数上涨，线程数不掉
压满 CPU	`--workload cpu`、线程数调到核数	吞吐曲线拉平、`avg_run` 上升
看 IO 型收益	`--workload io`、线程数远超核数	线程数多但吞吐仍上升（sleep 不占 CPU）

附录 B：实测数据

项目	数字	说明
全量重编	0 warning / 0 error	`-Wall -Wextra -Wpedantic -Wshadow -Wconversion` 全开
测试	**22 用例 / 136 断言 / 0 失败**	`tp_tests.exe` 与 `ctest` 一致；CTest 超时 60s
`snapshot()`（旧）	1340 µs/次（Debug, n=2048, k=64）	全量拷贝 + 全排序
`snapshot()`（新）	**104 µs/次**	≈ 12.8×；界面每帧从 1.34 ms 降到 0.10 ms
`rebuild()`（旧）	326 µs（Debug, n=2048）/ 30 µs（Release）	每 100ms 调一次 → 改为按老化步长限流，0.5s 一次
`rebuild()`（装饰版，已撤回）	Debug 快 1.5×，**Release 慢 36%**	L1 压力；注释保留在 `src/priority_queue.cpp`
CLI 冒烟	40 任务 / 2~8 线程	可见 `[ScaleUp] 扩容 +2 → 4 线程（积压 10）`、`[ScaleDown] 缩容 -2 → 2 线程`
合法字符输入	`--tasks -5 --batch abc`	回落默认 `tasks=300 batch=25`（修复前 `-5` 会回绕成 1.8e19）

> 基准脚本放在仓库之外（`%TEMP%\tfbench\`），用同一份 `src/priority_queue.cpp` 单独编译，只测算法本身。所以上面的数字是"算法成本"，不含线程调度噪声。

附录 C：一次任务的一生

```
① 提交（调用线程）
   submit(name, prio, work)
     ├─ work 为空 → return nullopt
     ├─ accepting() 为假 → return nullopt
     ├─ 建 Task（id/sequence 原子自增，state=Pending）
     └─ enqueue()
          ├─ 池锁内：queue_.push(task)
          │     └─ 队列满 → rejected_++ + Reject 事件 → return false → nullopt
          ├─ 池锁内：submitted_++ +（采样时）Submit 事件
          └─ 池锁外：cv_.notify_one()

② 出队（工作线程，池锁内）
   cv_.wait 醒来 → 检查 retire/stopping → queue_.pop()
     ├─ mark_busy(id)、state=Running、started_ns=now
     └─ running_tasks_[id] = task          ← 取消从这里能找到它
   解锁

③ 执行（工作线程，无锁）
   run_task()
     ├─ cancel_requested ? → Cancelled + 事件「开始执行前发现已取消」→ 返回
     ├─ started_++、累计等待时长、Start 事件（采样）
     ├─ try { work(); } catch(...) { ok = false; }        ← 异常隔离
     ├─ cancelled = cancel_observed（任务体是否真的看到取消）
     ├─ state = cancelled ? Cancelled : (ok ? Succeeded : Failed)
     ├─ mark_idle()、note_executed(run_ms)、run_ns_total_
     └─ 计数 + 事件（Finish 采样 / Fail 不采样 / Cancel 不采样）

④ 收尾（工作线程，池锁内）
   running_tasks_.erase(id)
   回到 ② 继续等下一个任务

⑤ 取消的三种情形（调用线程）
   A. 还在队列里：queue_.remove(id) 成功 → state=Cancelled、cancelled_++ → cancel() 返回 true
   B. 正在执行：running_tasks_ 命中且未结束 → 只置 cancel_requested → 返回 true
      （任务体若轮询 token → 自己 return → 记 Cancelled；若不理 → 跑完记 Succeeded）
   C. 已结束或不存在：两个容器都没有 → 记 Warn → 返回 false

⑥ 关闭（调用线程）
   shutdown(true)：停止收任务 → notify → 工作线程把队列吃干净后退出 → join（锁外）
   shutdown(false)：先把排队任务全标 Cancelled 并对在跑任务置取消请求 → 同上
```

结语

这个项目最值得带走的不是"线程池代码"，而是三样东西：

1. **职责边界**：内核不知道界面，界面不碰内部状态，组件之间互不认识。

于是队列可以单线程测、内核可以搬到别的平台、界面可以整个换掉。

2. **并发纪律**：锁内只搬数据、锁外执行与等待、先登记再暴露、取消靠协作。

这四条能挡住 90% 的多线程事故。

3. **可观测性**：`snapshot()` 值快照 + 环形事件日志，让"池子现在到底在干什么"随时可见。

没有观测的并发程序只能靠猜。

需要继续扩展时，最自然的几个方向：任务依赖（DAG）、定时任务、按任务分组的限流、把 `execute` 换成 fiber/协程、或者给 CLI 加 JMH 式的延迟分位统计（P50/P99）。